SPOŁECZNA AKADEMIA NAUK

www.san.edu.pl

Kierunek studiów: **Informatyka**

Nazwisko i Imię: Guzenda Krzysztof

Numer albumu: 104382

Semestr: **I**

nr grupy: 2

Studia magisterskie uzupełniające 2 stopnia

Projekt zaliczeniowy z przedmiotu

Analiza i Modelowanie Systemów Informatycznych

pt.” Projekt, specyfikacja oraz implementacja aplikacji webowej wspomagającej pracę Zespołów Ratownictwa Medycznego w ramach systemu PRM”

Prowadzący zajęcia: ***dr inż. Andrzej Rychlik***

Łódź 2024/2025

SPIS TREŚCI

Spis treści

1. Wprowadzenie 4
1.1. Cel 4
1.2. Zakres 4
1.3. Definicje, akronimy i skróty 5
1.4. Ogólny opis 5
2. Wymagania ogólne 6
2.1. Zainteresowane strony 6
2.2. Środowisko operacyjne 7
3. Wymagania szczegółowe 8
3.1. Funkcjonalności 8
3.2. Wymagania niefunkcjonalne 9
3.2.1. Bezpieczeństwo 9
3.2.2. Wydajność 9
3.2.3. Trafność 9
4. Interfejsy 10
4.1. Interfejs użytkownika 10
4.2. Interfejsy zewnętrzne 10
5. Ograniczenia 11
5.1. Technologiczne 11
5.2. Użytkowe 11
6. Wymagania dotyczące dokumentacji 12
6.1. Dokumentacja użytkownika 12
6.2. Dokumentacja deweloperska 12
6.3. Wymagania dotyczące wersjonowania dokumentacji 12
7. Harmonogram i budżet 13
7.1. Harmonogram 13
7.2. Budżet 13
8. Diagramy 16
8.1. Diagram klas 16

8.2. Diagram związków między encjami 17
8.3. Diagram sekwencji 17
8.4. Diagram aktywności 18
8.5. Diagram komponentów 19
8.6. Diagram wdrożeniowy 20
8.7. Diagram przypadków użycia 20
9. Projekt oraz implementacja prototypu aplikacji webowej 23
9.1. Podstawowe założenia projektu 23
9.2. Projekt oraz struktura bazy danych 29
9.3. API systemu 33
9.4. Graficzny interfejs użytkownika 40

1. Wprowadzenie

1.1. Cel

Celem projektu jest stworzenie aplikacji webowej, która wspiera Zespoły Ratownictwa Medycznego (ZRM) w ramach Państwowego Ratownictwa Medycznego (PRM). Aplikacja ma na celu:

- **Usprawnienie procesu zarządzania zgłoszeniami ratunkowymi**: Automatyzacja rejestracji zgłoszeń oraz ich przypisywania do odpowiednich zespołów.
- **Poprawę komunikacji**: Umożliwienie szybkiej wymiany informacji między dyspozytorami a zespołami ratowników za pomocą wbudowanego systemu powiadomień i czatu.
- **Monitorowanie działań w czasie rzeczywistym**: Śledzenie lokalizacji zespołów ratunkowych oraz statusu zgłoszeń.
- **Zarządzanie dokumentacją medyczną**: Umożliwienie ratownikom dodawania notatek o stanie pacjenta oraz przechowywania historii leczenia.
- **Generowanie raportów**: Tworzenie raportów dotyczących działań ratunkowych na podstawie zebranych danych.

1.2. Zakres

Zakres projektu obejmuje:

- **Analizę potrzeb użytkowników**: Przeprowadzenie wywiadów i ankiet wśród ratowników i dyspozytorów w celu zrozumienia ich wymagań.
- **Projektowanie interfejsu użytkownika**: Opracowanie prototypów UI/UX, które będą testowane przez użytkowników końcowych.
- **Implementację funkcjonalności**: Rozwój modułów aplikacji zgodnie z wymaganiami funkcjonalnymi i niefunkcjonalnymi.
- **Integrację z systemem PRM**: Zapewnienie współpracy z istniejącymi systemami ratowniczymi za pomocą API.

- **Testowanie i wdrożenie**: Przeprowadzenie testów jednostkowych, integracyjnych oraz akceptacyjnych przed wdrożeniem do użytku.
- **Szkolenie użytkowników**: Przygotowanie materiałów szkoleniowych oraz przeprowadzenie sesji szkoleniowych dla personelu.

1.3. Definicje, akronimy i skróty

- **PRM** – Państwowe Ratownictwo Medyczne: system organizacji pomocy medycznej w nagłych przypadkach.
- **ZRM** – Zespół Ratownictwa Medycznego: jednostka operacyjna PRM odpowiedzialna za udzielanie pomocy na miejscu zdarzenia.
- **API** – Interfejs Programowania Aplikacji (ang. Application Programming Interface): zestaw narzędzi umożliwiających integrację różnych systemów informatycznych.
- **UI/UX** – Interfejs Użytkownika / Doświadczenie Użytkownika (ang. User Interface / User Experience): projektowanie wizualne i funkcjonalne aplikacji dla użytkowników końcowych.
- **RODO** – Rozporządzenie o Ochronie Danych Osobowych: regulacje dotyczące ochrony danych osobowych w UE.

1.4. Ogólny opis

Aplikacja webowa ma na celu centralizację informacji o zgłoszeniach ratunkowych, umożliwiając ich efektywne zarządzanie oraz szybką wymianę danych między dyspozytorami a zespołami ratunkowymi. Dzięki zastosowaniu nowoczesnych technologii, aplikacja będzie dostępna na różnych urządzeniach mobilnych i stacjonarnych, co zwiększy jej użyteczność w codziennej pracy.

2. Wymagania ogólne

2.1. Zainteresowane strony

1. **Ratownicy medyczni**:

- Główna grupa użytkowników aplikacji pracujących w terenie.

- Potrzeby:

- - Szybki dostęp do informacji o zgłoszeniach.
- - Możliwość rejestrowania danych o stanie pacjenta w czasie rzeczywistym.
- - Intuicyjny interfejs dostosowany do pracy w trudnych warunkach terenowych.

2. **Dyspozytorzy medyczni**:

- Osoby koordynujące działania ratunkowe z dyspozytorni.

- Potrzeby:

- - Możliwość przypisywania zgłoszeń do odpowiednich zespołów na podstawie ich lokalizacji i dostępności.
- - Monitorowanie statusu zgłoszeń oraz lokalizacji zespołów w czasie rzeczywistym.

3. **Administratorzy systemu**:

- Osoby odpowiedzialne za zarządzanie aplikacją od strony technicznej i organizacyjnej.

- Potrzeby:

- - Narzędzia do konfiguracji systemu i zarządzania użytkownikami.
- - Dostęp do raportów dotyczących działania aplikacji oraz jej wydajności.

4. **Lekarze specjaliści**:

- Konsultanci korzystający z dokumentacji medycznej pacjentów w celu oceny przypadków lub dalszego leczenia.

2.2. Środowisko operacyjne

Aplikacja będzie działać w środowisku webowym, co oznacza:

- - Dostępność na przeglądarkach internetowych (Chrome, Firefox, Safari).
- - Możliwość używania na różnych urządzeniach: komputerach stacjonarnych, laptopach, tabletach i smartfonach.
- - Wymagania dotyczące stabilnego połączenia internetowego dla synchronizacji danych.

3. Wymagania szczegółowe

3.1. Funkcjonalności

Aplikacja powinna oferować następujące funkcjonalności:

1. **Zarządzanie zgłoszeniami**:

- Rejestracja nowych zgłoszeń przez dyspozytorów z możliwością przypisania priorytetu (np., „wysoki", „średni").
- Aktualizacja statusu zgłoszenia (np., „w drodze", „na miejscu", „zakończone").
- Możliwość dodawania notatek przez ratowników dotyczących stanu pacjenta.

2. **Lokalizacja zespołów ratunkowych**:

- Śledzenie pozycji zespołów na mapie interaktywnej za pomocą GPS.
- Wyświetlanie najbliższych punktów pomocy medycznej na mapie wraz z czasem dojazdu.

3. **Dokumentacja medyczna pacjentów**:

- Możliwość dodawania notatek o stanie pacjenta przez ratowników podczas interwencji.
- Przechowywanie historii leczenia pacjenta oraz danych kontaktowych szpitala docelowego.

4. **Generowanie raportów i statystyk**:

- Tworzenie raportów dotyczących działań ratunkowych na podstawie zebranych danych (np., liczba zgłoszeń, czas reakcji).
- Analiza statystyk dotyczących efektywności działań ZRM.

5. **Komunikacja między zespołami a dyspozytornią**:

- Wbudowany system czatu lub powiadomień umożliwiający szybką wymianę informacji między dyspozytorami a zespołami ratunkowymi.

3.2. Wymagania niefunkcjonalne

3.2.1. Bezpieczeństwo

Aplikacja musi zapewnić:

- Szyfrowanie danych przesyłanych przez sieć (SSL/TLS).
- Ochronę danych osobowych zgodnie z RODO.
- Mechanizmy autoryzacji i uwierzytelniania użytkowników (np., logowanie dwuskładnikowe).

3.2.2. Wydajność

Aplikacja powinna:

- Obsługiwać jednocześnie co najmniej 100 aktywnych użytkowników bez spadku wydajności.
- Zapewniać czas odpowiedzi na zapytania nieprzekraczający 2 sekund dla operacji krytycznych (np., rejestracja zgłoszenia).

3.2.3. Trafność

Aplikacja musi zapewnić:

- Dokładność lokalizacji zespołów ratunkowych z dokładnością do 5 metrów przy użyciu technologii GPS.
- Precyzyjne rejestrowanie danych o zgłoszeniach i ich statusach.

4. Interfejsy

4.1. Interfejs użytkownika

Interfejs powinien być:

- - Intuicyjny i przyjazny dla użytkownika, z jasnym podziałem na sekcje (zgłoszenia, lokalizacja, dokumentacja).
- - Responsywny, aby dostosowywał się do różnych rozmiarów ekranów urządzeń mobilnych i stacjonarnych.

Elementy interfejsu użytkownika:

- - Menu główne z łatwym dostępem do wszystkich funkcji aplikacji.
- - Powiadomienia o nowych zgłoszeniach lub aktualizacjach statusu zgłoszeń.
- - Mapa interaktywna pokazująca lokalizację zespołów i punktów pomocy.

4.2. Interfejsy zewnętrzne

Aplikacja powinna mieć możliwość integracji z innymi systemami poprzez API, co umożliwi:

- - Wymianę danych o zgłoszeniach z systemami szpitalnymi (np., EHR – Electronic Health Record).
- - Synchronizację lokalizacji zespołów z systemami GPS używanymi przez służby ratunkowe.

Specyfikacje API:

- - Endpointy do rejestracji zgłoszeń, aktualizacji statusów oraz pobierania informacji o lokalizacji zespołów.
- - Dokumentacja API zawierająca przykłady zapytań oraz odpowiedzi serwera.

5. Ograniczenia

5.1. Technologiczne

Ograniczenia mogą obejmować:

- Wymagania dotyczące sprzętu (np., urządzenia mobilne muszą obsługiwać najnowsze wersje przeglądarek).

5.2. Użytkowe

Ograniczenia związane z użytecznością mogą obejmować:

- Potrzebę przeszkolenia użytkowników końcowych w zakresie obsługi aplikacji.

Potencjalne trudności:

- Różnice w poziomie umiejętności technologicznych użytkowników mogą wpłynąć na efektywność korzystania z aplikacji.

6. Wymagania dotyczące dokumentacji

6.1. Dokumentacja użytkownika

Dokumentacja powinna zawierać:

- Instrukcje obsługi aplikacji dla różnych grup użytkowników (ratownicy, dyspozytorzy), obejmujące wszystkie funkcje aplikacji oraz przykłady użycia.

6.2. Dokumentacja deweloperska

Dokumentacja techniczna dla programistów powinna zawierać:

- Opis architektury systemu oraz używanych technologii (np., języki programowania, frameworki).

Dodatkowe informacje:

- Wytyczne dotyczące rozwoju kodu źródłowego oraz standardy kodowania stosowane w projekcie.

6.3. Wymagania dotyczące wersjonowania dokumentacji

Wszystkie dokumenty powinny być wersjonowane zgodnie z ustalonymi standardami, aby zapewnić aktualność informacji i umożliwić śledzenie zmian w projekcie.

7. Harmonogram i budżet

7.1. Harmonogram

Harmonogram prac powinien zawierać następujące fazy:

Faza	Czas trwania
Analiza wymagań	2 tygodnie
Projektowanie	4 tygodnie
Implementacja	6 tygodni
Testowanie	3 tygodnie
Wdrożenie	1 tydzień

Szczegóły faz harmonogramu:

1. **Analiza wymagań**: Spotkania z interesariuszami oraz zbieranie wymagań funkcjonalnych i niefunkcjonalnych.
2. **Projektowanie**: Opracowanie prototypów UI/UX oraz architektury systemu; przeprowadzenie testów prototypów przez użytkowników końcowych.
3. **Implementacja**: Rozwój modułów aplikacji zgodnie z wymaganiami; regularne spotkania zespołu deweloperskiego w celu omówienia postępów prac.
4. **Testowanie**: Przeprowadzenie testów jednostkowych, integracyjnych oraz akceptacyjnych; zbieranie feedbacku od użytkowników beta-testujących aplikację przed pełnym wdrożeniem.
5. **Wdrożenie**: Przeniesienie aplikacji na środowisko produkcyjne; szkolenie użytkowników końcowych.

7.2. Budżet

Budżet projektu powinien obejmować:

1. Koszty wynagrodzeń zespołu deweloperskiego (programiści, projektanci UI/UX).
2. Koszty sprzętu i oprogramowania potrzebnego do rozwoju aplikacji (serwery, licencje).

3. Koszty szkoleń dla użytkowników końcowych oraz materiałów edukacyjnych.

1. Koszty wynagrodzeń zespołu deweloperskiego

- **Programiści**:
  - 2 programistów backendowych: 10 000 PLN/miesiąc x 6 miesięcy = 120 000 PLN
  - 2 programistów frontendowych: 9 000 PLN/miesiąc x 6 miesięcy = 108 000 PLN
- **Projektant UI/UX**:
  - 1 projektant: 8 000 PLN/miesiąc x 4 miesiące = 32 000 PLN
- **Testerzy**:
  - 2 testerów: 7 000 PLN/miesiąc x 3 miesiące = 42 000 PLN

Razem koszty wynagrodzeń:

- Programiści: 120 000 + 108 000 = 228 000 PLN
- Projektant UI/UX: 32 000 PLN
- Testerzy: 42 000 PLN

Łącznie: 228 000 + 32 000 + 42 000 = 302 000 PLN

2. Koszty sprzętu i oprogramowania

- **Serwery**:
  - Koszt zakupu serwera dedykowanego: 15 000 PLN (jednorazowy)
  - Koszt hostingu (roczny): 5 000 PLN
- **Oprogramowanie**:
  - Licencje na oprogramowanie do zarządzania projektem (np. Jira): 2 500 PLN rocznie.
  - Licencje na narzędzia do testowania (np. Selenium): 1 500 PLN rocznie.

Razem koszty sprzętu i oprogramowania:

- Serwery: 15 000 + 5 000 = 20 000 PLN
- Oprogramowanie: 2 500 + 1 500 = 4 000 PLN

Łącznie: 20 000 + 4 000 = 24 000 PLN

3. Koszty szkoleń dla użytkowników

- **Szkolenia dla ratowników i dyspozytorów**:
  - Koszt szkolenia dla grupy (20 osób): 3 000 PLN za sesję x 3 sesje = 9 000 PLN
  - Materiały szkoleniowe (broszury, instrukcje): 1 500 PLN

Razem koszty szkoleń:

- Szkolenia: 9 000 PLN
- Materiały szkoleniowe: 1 500 PLN

Łącznie: 9 000 + 1 500 = 10 500 PLN

4. Koszty marketingowe i promocji

- **Promocja aplikacji wśród użytkowników końcowych**:
  - Kampania informacyjna (plakaty, ulotki): 5 000 PLN
  - Spotkania informacyjne z interesariuszami: 3 000 PLN

Razem koszty marketingowe i promocji:

- Kampania informacyjna: 5 000 PLN
- Spotkania informacyjne: 3 000 PLN

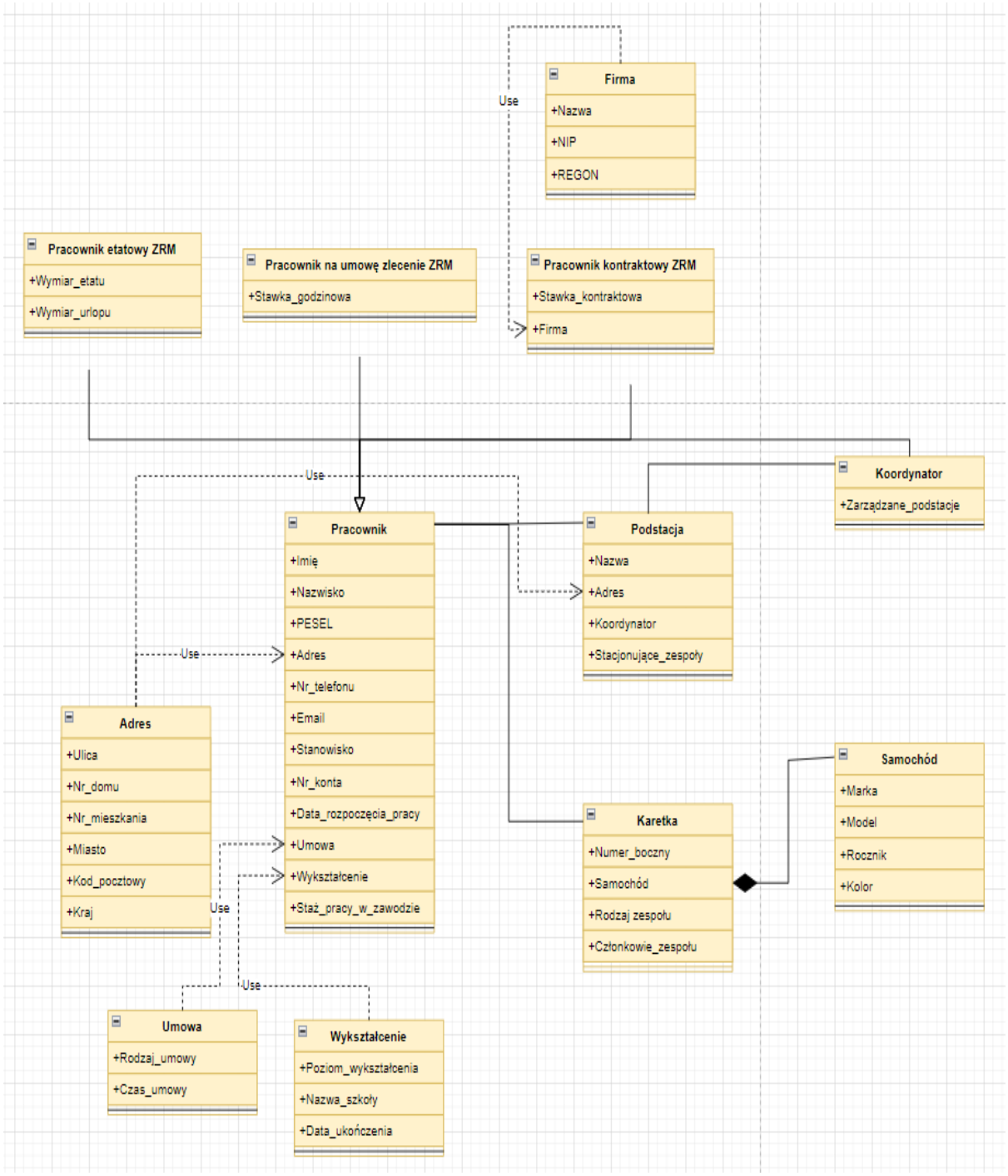
Łącznie: 5 000 + 3 000 = 8 000 PLN

Podsumowanie budżetu projektu

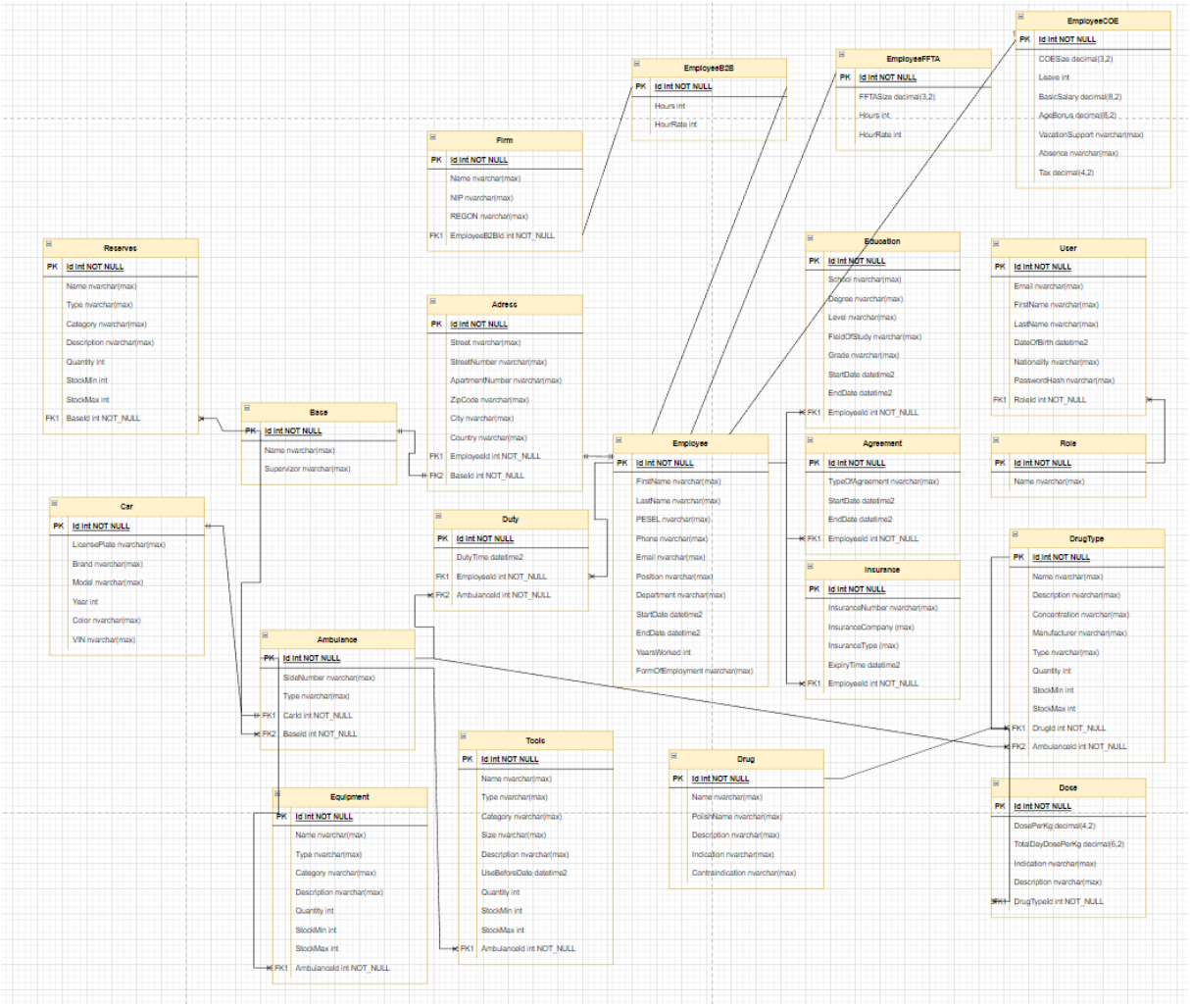
Kategoria	Koszt (PLN)
Wynagrodzenia	**302 000**
Sprzęt i oprogramowanie	**24 000**
Szkolenia	**10 500**
Marketing i promocja	**8 000**
Razem całkowity koszt projektu	**364 500**

8. Diagramy

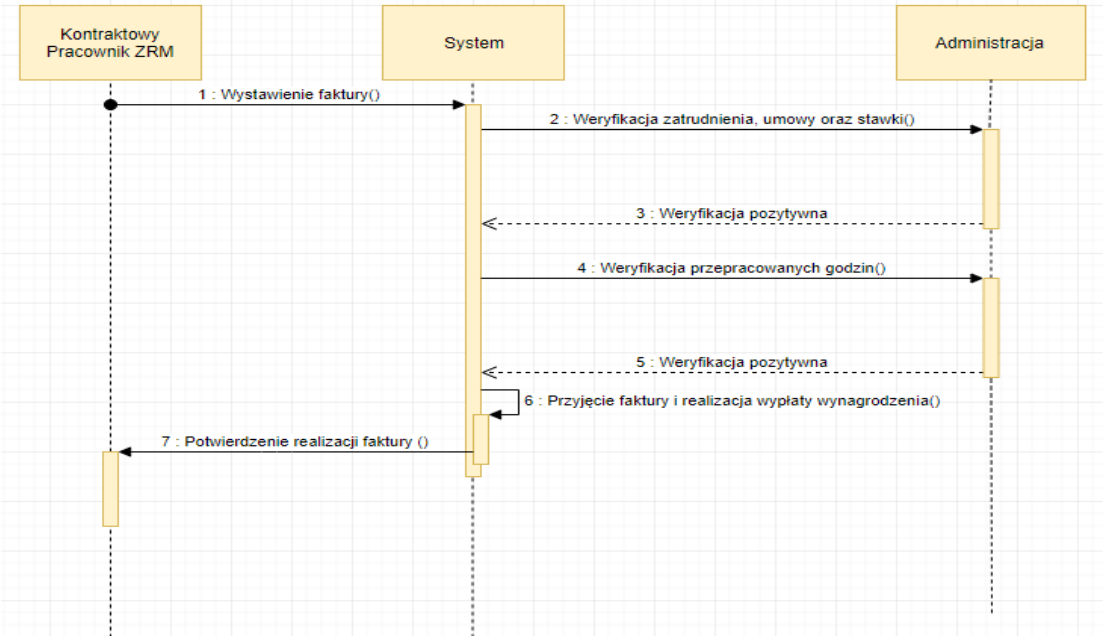
8.1. Diagram klas



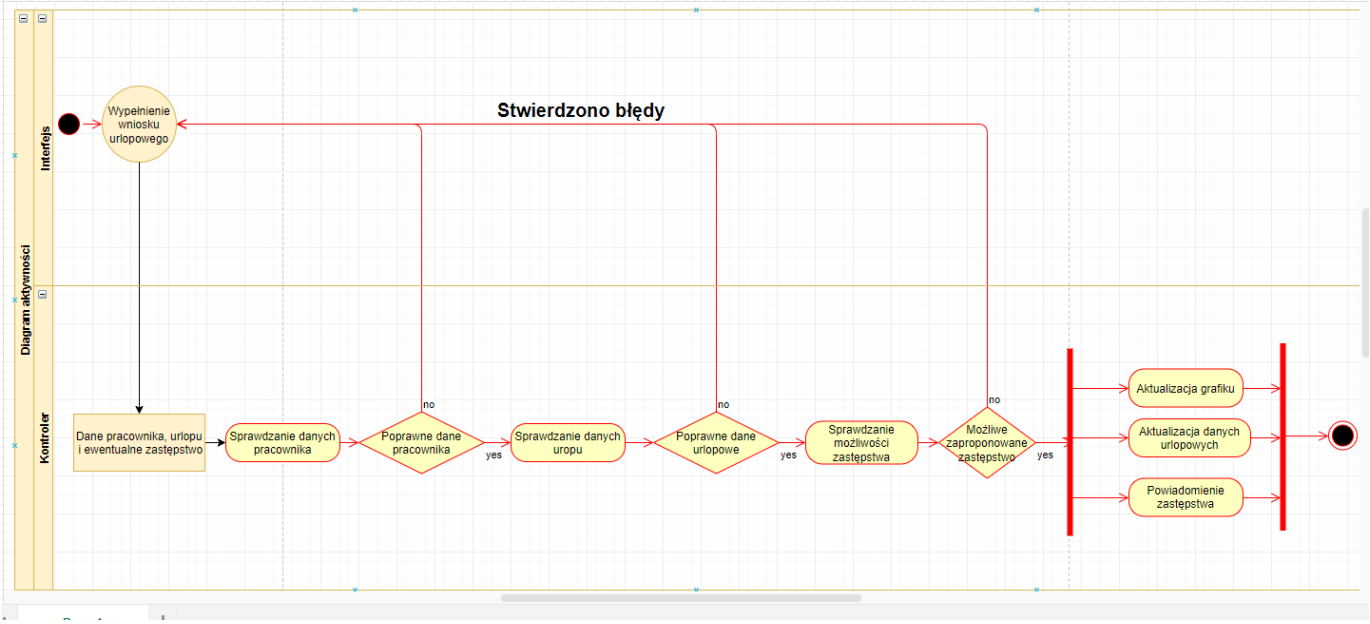
8.2. Diagram związków między encjami

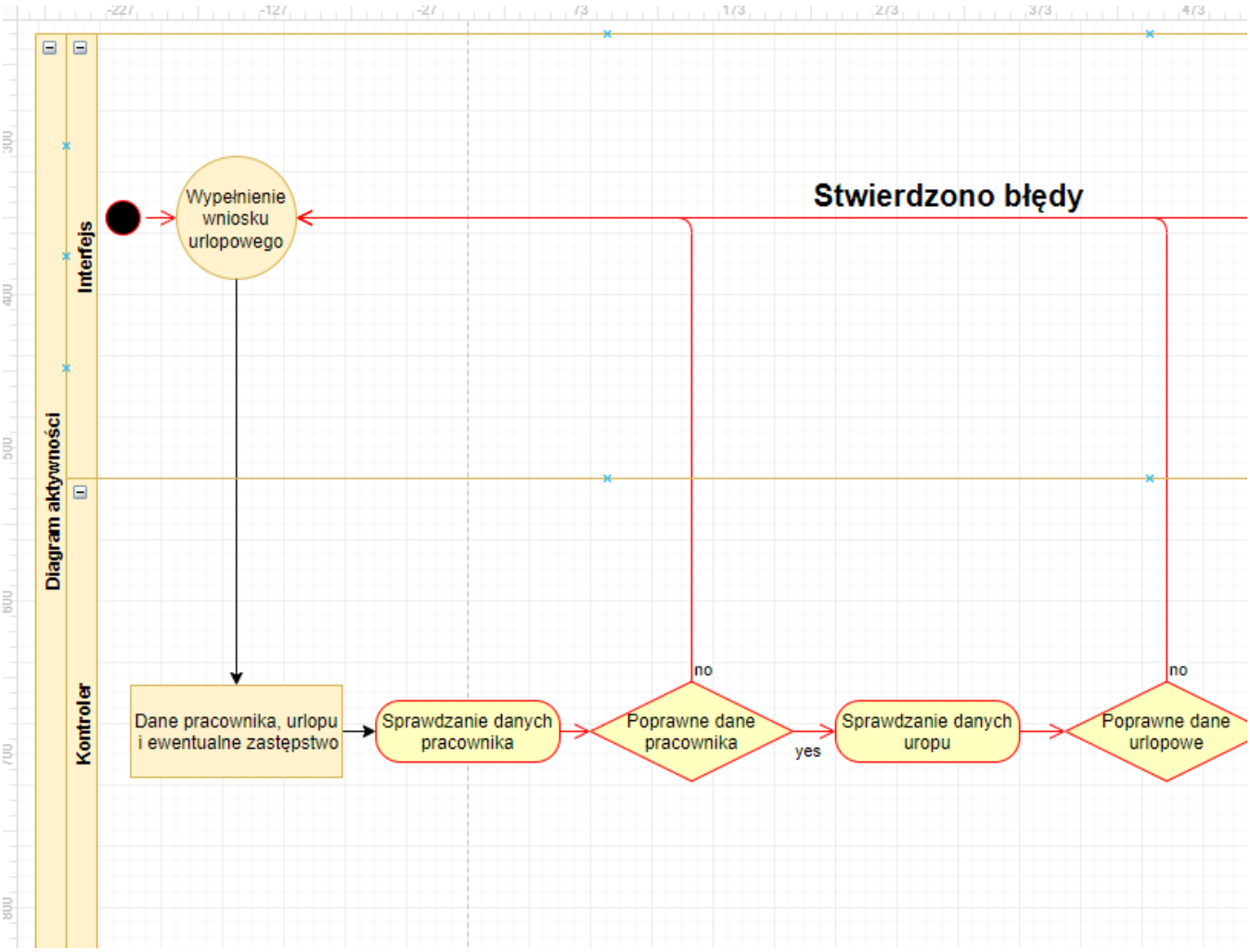


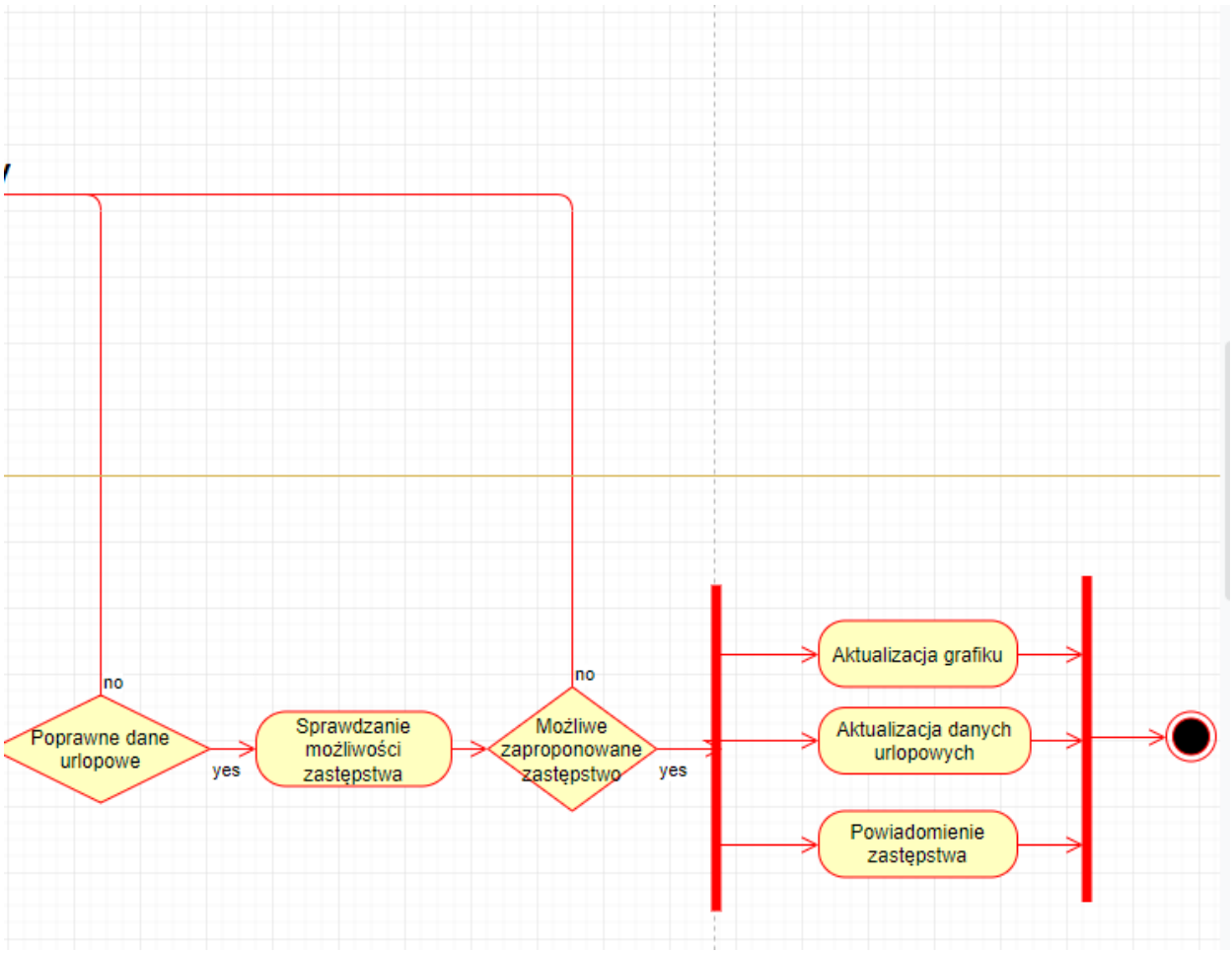
8.3. Diagram sekwencji



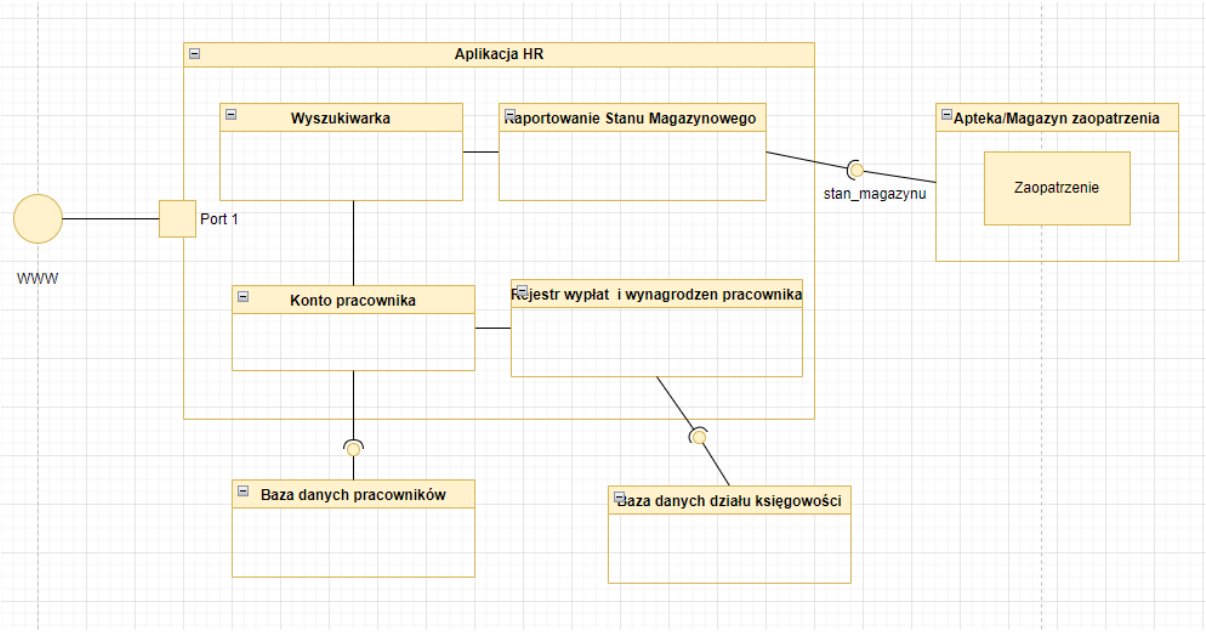
8.4. Diagram aktywności



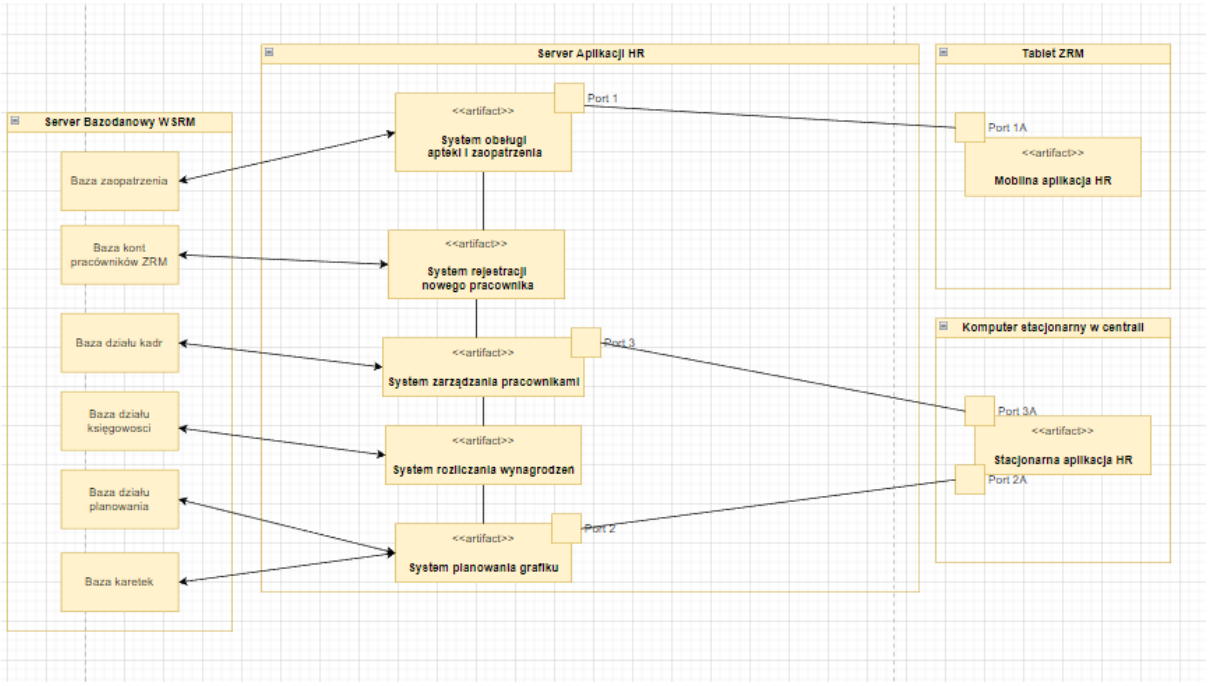




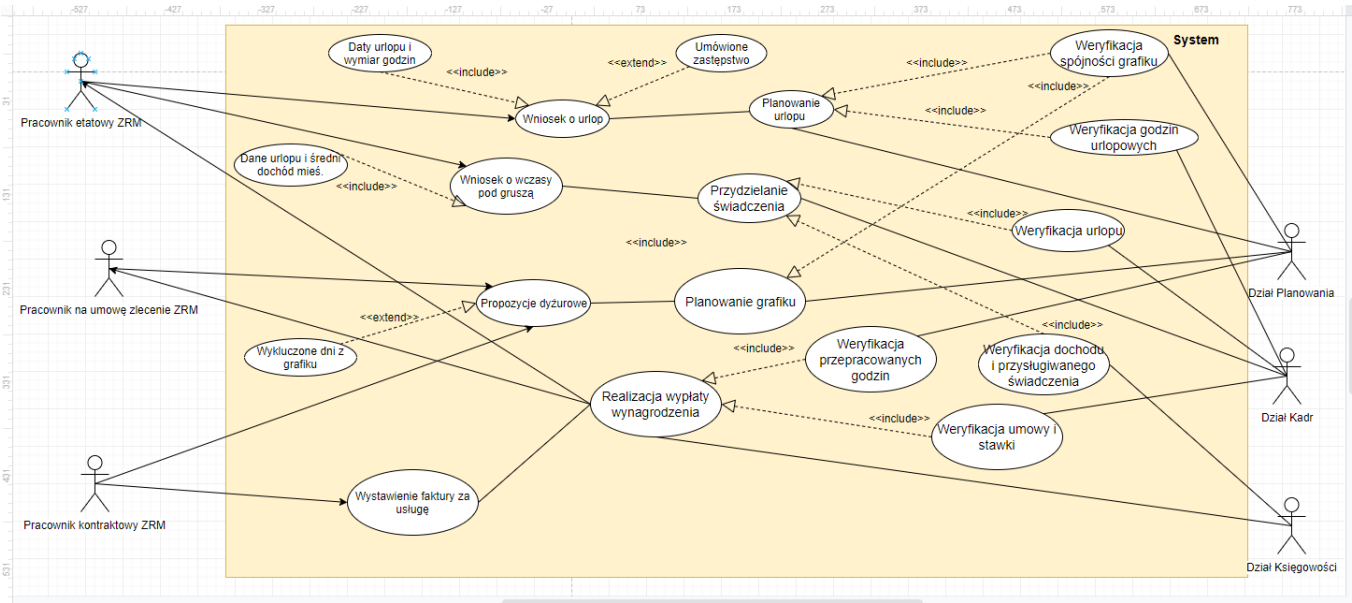
8.5. Diagram komponentów

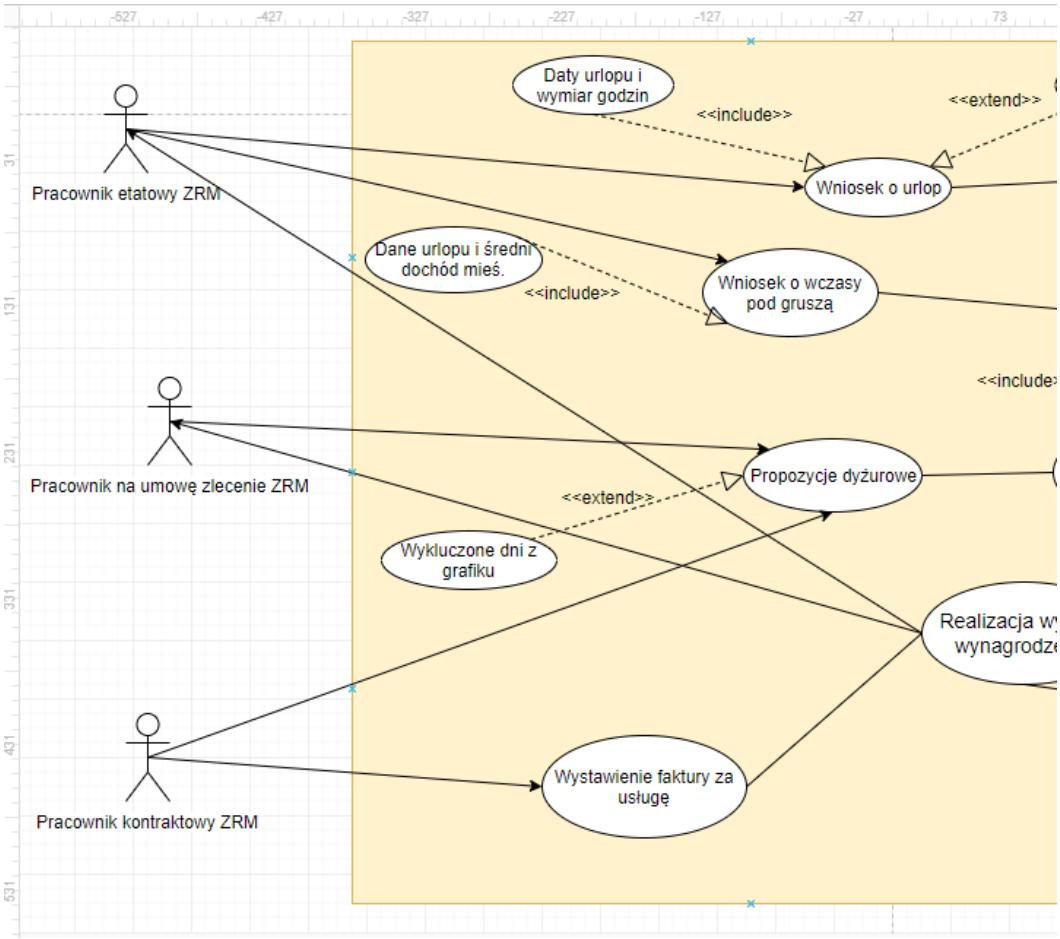


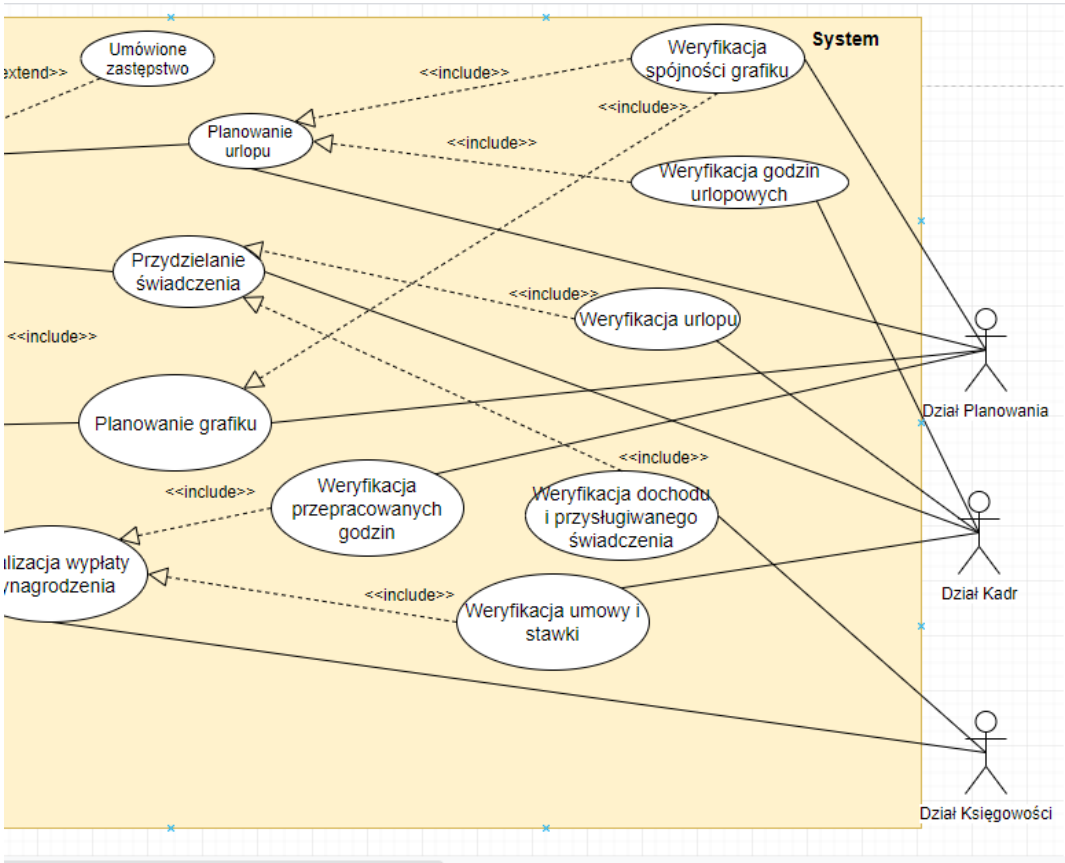
8.6. Diagram wdrożeniowy

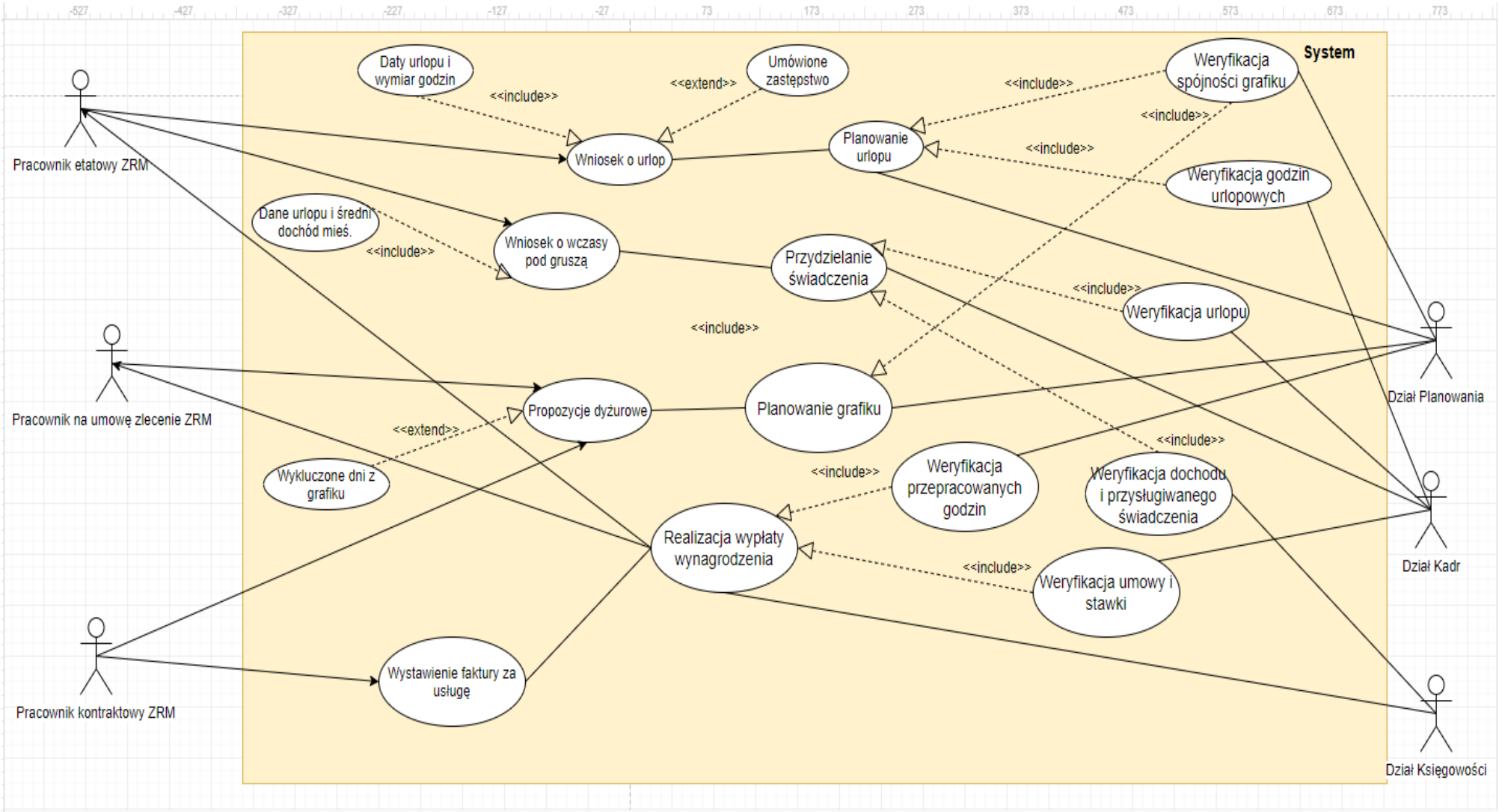


8.7. Diagram przypadków użycia




Daty urlopu i wymiar godzin
<<include>>
<<extend>>
Pracownik etatowy ZRM
Wniosek o urlop
Dane urlopu i średni dochód mieś.
<<include>>
Wniosek o wczasy pod gruszą
<<include>
Pracownik na umowę zlecenie ZRM
Propozycje dyżurowe
<<extend>>
Wykluczone dni z grafiku
Realizacja w
wynagrodz
Pracownik kontraktowy ZRM
Wystawienie faktury za usługę


System
Umówione zastępstwo
xtend>>
<<include>>
Weryfikacja spójności grafiku
<<include>>
Planowanie urlopu
<<include>>
Weryfikacja godzin urlopowych
Przydzielanie świadczenia
<<include>>
Weryfikacja urlopu
<<include>>
Dział Planowania
Planowanie grafiku
<<include>>
<<include>>
Weryfikacja przepracowanych godzin
Weryfikacja dochodu i przysługiwanego świadczenia
lizacja wypłaty
ynagrodzenia
<<include>>
Weryfikacja umowy i stawki
Dział Kadr
Dział Księgowości


System
Pracownik etatowy ZRM
Pracownik na umowę zlecenie ZRM
Pracownik kontraktowy ZRM
Dział Planowania
Dział Kadr
Dział Księgowości
Daty urlopu i wymiar godzin
Umówione zastępstwo
Weryfikacja spójności grafiku
Wniosek o urlop
Planowanie urlopu
Weryfikacja godzin urlopowych
Dane urlopu i średni dochód mieś.
Wniosek o wczasy pod gruszą
Przydzielanie świadczenia
Weryfikacja urlopu
Propozycje dyżurowe
Planowanie grafiku
Wykluczone dni z grafiku
Weryfikacja przepracowanych godzin
Weryfikacja dochodu i przysługiwanego świadczenia
Realizacja wypłaty wynagrodzenia
Weryfikacja umowy i stawki
Wystawienie faktury za usługę
<<include>>
<<extend>>

9. Projekt oraz implementacja prototypu aplikacji webowej

9.1. Podstawowe założenia projektu

Główną ideą projektu było stworzenie narzędzia, oprogramowania, które poprzez swoje funkcjonalności wspomagać mogłoby codzienną pracę członków zespołów ratownictwa medycznego w ramach Systemu Państwowe Ratownictwo Medyczne. Miało to być rozwiązanie współgrające z innymi już obecnie używanymi w środowisku medycznym, jak na przykład: System Wspomagania Dowodzenia Państwowego Ratownictwa Medycznego, SIRON, eMPendium mobile, DrWidget, NaviExpert Telematics, Gabinet.gov.pl, czy strona https://www.prc.krakow.pl/[1].

Po przeanalizowaniu większości z tych aplikacji można było wysunąć zdecydowany wniosek, iż znaczna część z nich to aplikacje webowe. Dlatego podczas podejmowania decyzji o rodzaju aplikacji wybór padł właśnie na aplikacje webowa. Dodatkowa aplikacja webowa daje możliwość korzystania z niej na dowolnie wybranym urządzeniu. Nie ma znaczenia, czy będzie to komputer z systemem Windows, Linux, MacOS, czy urządzenie mobilne oparte na systemie Android. Prostota w uruchamianiu oraz totalna wieloplatformowość to zdecydowanie najważniejsze zalety aplikacji webowych. Do prawidłowego funkcjonowania, praktycznie na każdym urządzeniu elektronicznym, wystarczy jedynie dostęp do Internetu oraz przeglądarka internetowa[2]. Do tego nie ma praktycznie żadnych wymagań technicznych co do przeglądarki. Obecnie funkcjonują tak zaawansowane standardy przeglądarek internetowych, że wybór jakiejkolwiek będzie prawidłowym wyborem. Pozostaje jedynie kwestia preferencji czy przyzwyczajenia[3].

[1] https://www.prc.krakow.pl/index.html [dostęp: 23.02.2023]

[2] https://www.empressia.pl/blog/72-aplikacje-webowe-czym-sa-jak-dzialaja-i-jakie-sa-ich-zalety [dostęp: 23.02.2023]

[3] https://pl.wikipedia.org/wiki/Por%C3%B3wnanie_przegl%C4%85darek_internetowych [dostęp: 23.02.2023]

Na dodatek wszystkiego aplikacja webowa nie wymaga konieczności instancji żadnego nowego oprogramowania na urządzenie (każde nowoczesne urządzenie ma domyślnie zainstalowaną jakąś przeglądarkę internetową), co może być klucze w przypadku urządzeń służbowych, z założonymi kontami pracowników jako użytkowników, z ograniczonymi prawami dostępu do niektórych funkcjonalności urządzenia bez poświadczenia hasłem administratora sytemu.

Aplikacja webowa, inaczej aplikacja internetowa, to pewnego rodzaju wersja strony internetowej rozszerzonej o funkcje interaktywne oraz o możliwość wprowadzania i publikowania treści na stronie przez użytkowników[4]. Najpopularniejszym przykładem takiej aplikacji webowej są serwisy społecznościowej Facebook czy Twitter, aplikacje aukcyjne Allegro, eBay, czy mapy online. Jest to taki rodzaj programu komputerowego, który wykorzystuje przeglądarkę internetowa do wyświetlania oraz wykonywania funkcjonalności graficznego interfejsu użytkownika aplikacji. Następnie za pomocą sieci komputerowej i przeglądarki host użytkownika komputera komunikuje się z serwerem aplikacji[5].

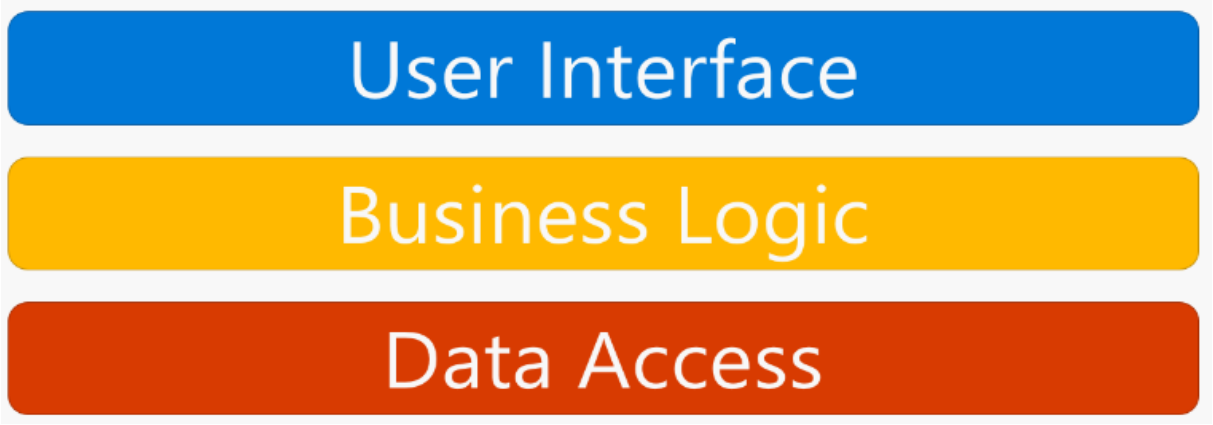
Stworzenie takie kompletnej oraz funkcjonalnej aplikacji webowej wymaga podzielenia projektu na dwie warstwy. Zgodnie z ogólnoświatowym standardem przyjmujemy warstwę „frontendową" jako tę pełniąca rolę graficznego interfejsu użytkownika oraz warstwę „backendową" odpowiedzialną za komunikacje z serwerem i zarządzanie bazą danych[6].

[4] https://digitallyher.pl/podstawowe-typy-aplikacji/ [dostęp: 23.02.2023]

[5] https://boringowl.io/blog/aplikacja-webowa-a-strona-internetowa [dostęp: 23.02.2023]

[6] https://kariera.comarch.pl/blog/czym-rozni-sie-fronted-od-backendu-i-kim-jest-frontend-developer/ [dostęp: 23.02.2023]

Rysunek 1. Warstwy aplikacji webowej



Źródło:https://learn.microsoft.com/pl-pl/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures [dostęp: 25.01.2023]

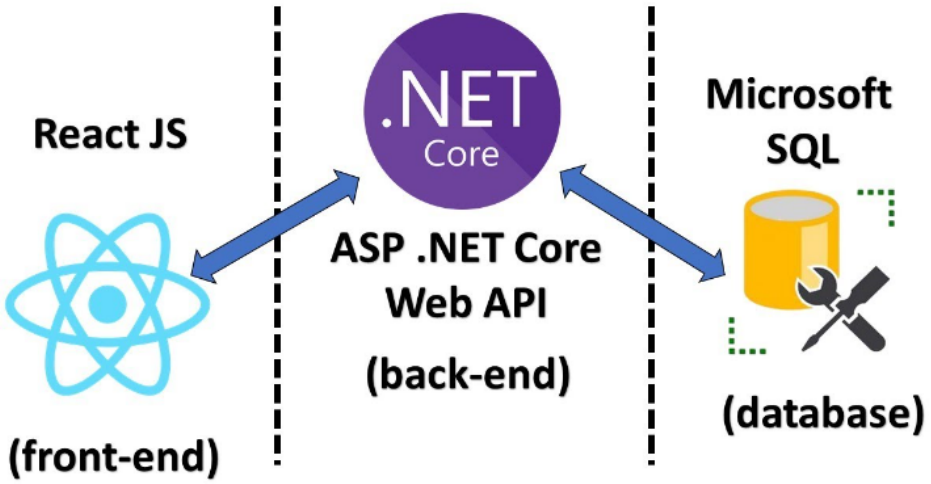
W monecie wyboru napisania projektu jako aplikacji webowej pozostaje tak naprawdę już tylko wybór odpowiedniej technologii. Zarówno dla warstwy „frontendowej", jak i „backendowej". Oczywistym wydaje się konieczność skorzystania z technologii języków HTML, CSS oraz JavaScript przy tworzeniu części „frontendowej" projektu, natomiast na poziomie obecnego zaawansowania w tworzeniu aplikacji webowych to podejście wydaje lekko przestarzałe. Te technologie stanowią idealna podstawę do projektu, ale należałoby jednak je rozwinąć o nowoczesne zdobycze technologiczne.

Idealnym przykładem takiej technologii dla warstwy „frontendowej" jest najpopularniejsza biblioteka języka JavaScript na świecie – React.js. React łączy ze sobą prostotę budowania aplikacji webowej za pomocą gotowych komponentów wraz z imponująca wydajnością i szerokim wsparciem technicznym społeczności[7]. Wraz z React w projekcie zastała zaimplementowana dodatkowo biblioteka komponentów React - Material UI.

[7] https://impicode.pl/blog/najpopularniejsze-frameworki-javascript/ [dostęp: 23.02.2023]

Jeśli chodzi natomiast o wybór technologii do części „backendowej” to zdecydowane popularną i na pewno sprawdzą opcją jest wybranie frameworku od firmy Microsoft, czyli ASP.NET, który jest częścią większej całości technologii .NET. Jest to jeden z popularniejszych wyborów od lat[8], a dodatkowo firma Microsoft nieprzerwanie rozwija ta technologię, dodając coraz to nowe funkcjonalności oraz dając gwarancję, iż .NET będzie jeszcze długo uczestniczył w tworzeniu oprogramowania[9].

Rysunek 2. Uproszczony model architektury rozwiązania aplikacji



Źródło: https://www.youtube.com/watch?v=ON-Z1iD6Y-c&ab_channel=ArtofEngineer [dostęp: 25.01.2023]

Jako wybór narzędzi do tworzenia oprogramowania w projekcie wykorzystany został Visual Studio Code, jako lekki edytor tekstu warstwy „frontendowej”. Nad wyborem przesadziła rozbudowana baza pakietów i rozszerzeń tego edytora. Do warstwy „backendowej” użyty został Visual Studio 2022 w wersji Community od firmy Microsoft. To zintegrowane środowisko programistyczne dostarcza wszelkie, potrzebne oraz zaawanasowane funkcjonalności do sprawnego tworzenia „backendowej” warstwy aplikacji webowej.

Do zarządzania i uzyskiwania dostępu do bazy danych zastosowany został program SQL Server Management Studio w wersji 18, również od firmy Microsoft. Firma Microsoft dostarcza tak naprawdę kompletne środowisko do tworzenia aplikacji webowych na każdym jej etapie.

[8] https://survey.stackoverflow.co/2022/#most-loved-dreaded-and-wanted-language-love-dread [dostęp: 23.02.2023]

[9] https://codenga.pl/artykuly/poradniki/csharp_i_net_jakie_maja_znaczenie_w_roku_2023 [dostęp: 23.02.2023]

W celu poprawnego przygotowania środowiska do pracy przy projekcie należy:

- pobrać oraz zainstalować:
    - framework .NET SDK[10],
    - środowisko uruchomieniowe języka JavaScript – Node.js[11],
    - program Visual Studio Code[12], Visual Studio Community[13],
    - SQL Server Express i SQL Server Management Studio [14],
- stworzyć w Visual Studio szablon projektu aplikacji webowej React z wykorzystaniem ASP.NET jako API[15],
- dodać do projektu:
    - początkową bibliotekę pakietów[16],
    - bibliotekę komponentów React – Material UI[17].

Aplikacja webowa, nazwana „ZRM Help", z założenia miała być minimalistyczną aplikacją webową, dostępną tylko po wcześniejszym zalogowaniu się i potwierdzeniu tożsamości. Korzystać z niej mają nie tylko ratownicy medyczni, ale i przede wszystkim koordynatorzy oraz kierownicy. Koniecznie przygotowanie będzie funkcjonalności rozróżniania logowania jako użytkownik lub jako administrator. Na stronie powinna też być możliwość zarejestrowania się jako nowy użytkownik, z ewentualnym późniejszym e-mailem w celu potwierdzenia tożsamości. Po analizie i charakterystyce podobnych aplikacji webowych stosowne wydaje się zastosowanie nagłówka i stopki strony. Warto jednak pozostawić ich minimalistyczna, ale symboliczną obecność.

---

[10] https://dotnet.microsoft.com/en-us/download [dostęp: 23.02.2023]

[11] https://nodejs.org/en [dostęp: 23.02.2023]

[12] https://code.visualstudio.com/ [dostęp: 23.02.2023]

[13] https://visualstudio.microsoft.com/pl/ [dostęp: 23.02.2023]

[14] https://www.microsoft.com/pl-pl/sql-server/sql-server-downloads [dostęp: 23.02.2023]

[15] https://learn.microsoft.com/pl-pl/visualstudio/javascript/tutorial-asp-net-core-with-react?source=recommendations&view=vs-2022 [dostęp: 23.02.2023]

[16] https://docs.npmjs.com/cli/v8/commands/npm-install [dostęp: 23.02.2023]

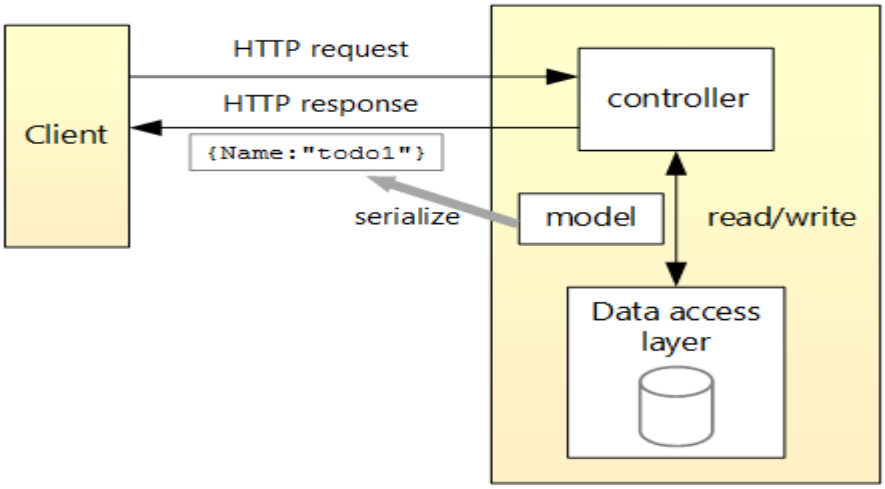
[17] https://mui.com/ [dostęp: 23.02.2023]

Spośród najważniejszych funkcjonalności wyświetlanych po stornie „frontendowej” aplikacji webowej „ZRM Help” należy wyróżnić:

- nagłówek strony oraz stopka strony,
- formularz logowania i formularz rejestracji,
- różnicowanie logowania przez użytkownika i administratora,
- na stronie głównej projektu różne kalkulatory medyczne przeliczające dawki leków, rozmiary sprzętu oraz wykonujące wszelkie inne niezbędne obliczenia,
- zakładka do zarządzania lekami, sprawdzania dawkowania, wskazań przeciwskazań oraz potencjalnie wielu innych opcji leków i preparatów medycznych,
- zakładka do zarzadzania stanem magazynowym produktów, preparatów i sprzętu medycznego w magazynie i karetkach zespołowych,
- po zalogowaniu się jako administrator dostępna baza pracowników do zarządzania pracownikami, składami karetek, podstacjami, dokumentami pracowników.

Na stronie „backendowej” projektu koniecznie było zaprojektowanie kilkunastu klas w języku C#. wraz z zaprogramowaniem ich kontekstu. Następnie wykonano kontrolery dla wszystkich klas w celu zarządzania ich zasobami i tabelami w bazie danych. W celu poprawnego zaprogramowania zależności między encjami w projekcie użyto mapper obiektowo-reakcyjny języka C#, jakim jest Entity Framework Core, za pomocą to którego następnie wykonanie migracje tworzącą kompletna bazę danych. Połącznie z baza danych zostało odpowiednio wcześniej zdefiniowane.

Rysunek 3. Diagram architektury rozwiązania aplikacji webowej

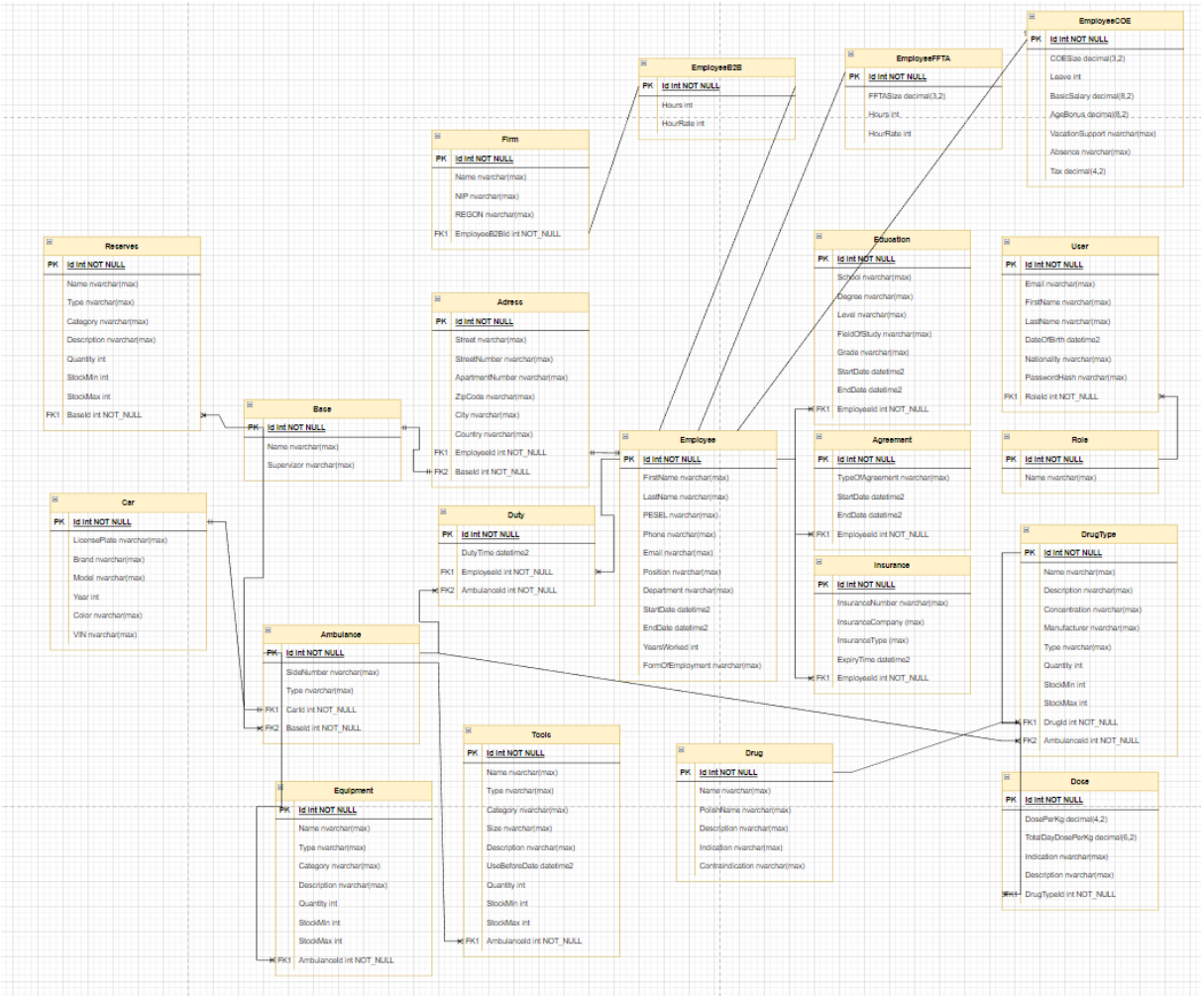


Źródło: https://learn.microsoft.com/pl-pl/aspnet/core/tutorials/first-web-api?view=aspnetcore-7.0&tabs=visual-studio [dostęp: 25.01.2023]

9.2. Projekt oraz struktura bazy danych

Projektowanie bazy danych należałoby zacząć od zastanowienia się nad funkcjonalnościami naszego projektu. Każda później zrealizowana klasa będzie przecież pełnić jakieś funkcje, czy wykonywać określone zadania. Trzymanie w projekcie pustych i bezużytecznych klas uznawane jest powszechnie za złą praktykę. W pracy zaprojektowanych zostało 25 unikatowych klas w języku C#, które powstały z myślą realizowania konkretnych funkcjonalności.

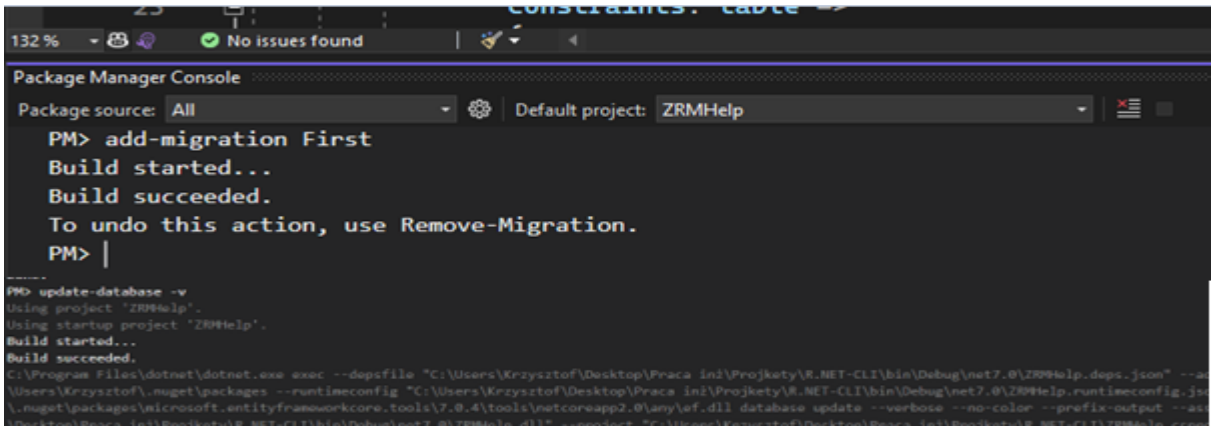
Rysunek 4. Diagram związków między encjami bazy danych



Źródło: repozytorium projektu aplikacji webowej

Cały proces tworzenia oraz projektowania bazy danych w projekcie aplikacji webowej z wykorzystaniem ASP.NET, dzięki wykorzystaniu Entity Framework Core (EF Core), nie wymaga od tworzącego żadnej znajomości języka SQL. Wszystkie polecenia i instrukcje pisane są za pomocą języka C# oraz .NET, a EF Core dzięki swoim zależnością tłumaczy ten kod na SQL.

Rysunek 5. Komendy EF Core w konsoli menadżera pakietów NuGet



Źródło: repozytorium projektu aplikacji webowej „ZRM Help”

Komenda "add-migration" jest używana w .NET EF Core do generowania migracji bazy danych na podstawie zmian w modelu bazy danych. Migracje to automatycznie wygenerowany kod, który reprezentuje zmiany w modelu bazy danych. Na przykład, dodanie lub usunięcie encji, właściwości, indeksów, relacji między encjami, itp. Każda migracja jest przechowywana w osobnym pliku, który zawiera kod do zastosowania zmian w bazie danych.

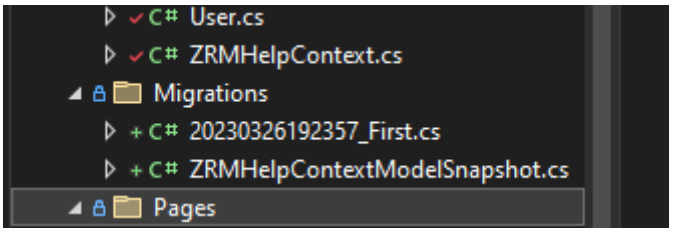
Komenda update-database -v jest używana do zastosowania migracji do bazy danych. Opcja -v (verbose) służy do wyświetlania szczegółowych informacji o procesie aktualizacji bazy danych. Podczas wykonywania tej komendy, EF Core porównuje listę migracji w plikach z migracjami z listą migracji już zastosowanych do bazy danych. Następnie, jeśli istnieją jakieś brakujące migracje, EF Core stosuje je do bazy danych w kolejności, w jakiej zostały utworzone.

Rysunek 6. Fragment przykładowej migracji EF Core projektu

```
51                  .Annotation("SqlServer:Identity", "1, 1"),
52              Name = table.Column<string>(type: "nvarchar(max)", nullable: true),
53              PolishName = table.Column<string>(type: "nvarchar(max)", nullable: true),
54              Description = table.Column<string>(type: "nvarchar(max)", nullable: true),
55              Indication = table.Column<string>(type: "nvarchar(max)", nullable: true),
56              Contraindication = table.Column<string>(type: "nvarchar(max)", nullable: true)
57          },
58          constraints: table =>
59          {
60              table.PrimaryKey("PK_Drugs", x => x.Id);
61          });
62
63      migrationBuilder.CreateTable(
64          name: "Employees",
65          columns: table => new
66          {
67              Id = table.Column<int>(type: "int", nullable: false)
68                  .Annotation("SqlServer:Identity", "1, 1"),
69              FirstName = table.Column<string>(type: "nvarchar(100)", maxLength: 100, nullable: true),
70              LastName = table.Column<string>(type: "nvarchar(max)", nullable: true),
71              NumerPESEL = table.Column<string>(name: "Numer PESEL", type: "nvarchar(max)", nullable: false),
72              Phone = table.Column<string>(type: "varchar(9)", nullable: true),
73              Email = table.Column<string>(type: "nvarchar(max)", nullable: true),
74              Position = table.Column<string>(type: "nvarchar(max)", nullable: true),
75              Department = table.Column<string>(type: "nvarchar(max)", nullable: true),
76              StartDate = table.Column<DateTime>(type: "datetime2", nullable: false),
77              EndDate = table.Column<DateTime>(type: "datetime2", nullable: true),
78              YearsWorked = table.Column<int>(type: "int", nullable: false, defaultValue: 0),
79              FormOfEmployment = table.Column<string>(type: "nvarchar(max)", nullable: true),
80              Discriminator = table.Column<string>(type: "nvarchar(max)", nullable: false),
81              Hours = table.Column<int>(type: "int", nullable: true),
82              HourRate = table.Column<int>(type: "int", nullable: true),
83              COESize = table.Column<decimal>(type: "decimal(3,2)", precision: 3, scale: 2, nullable: true),
84              Leave = table.Column<int>(type: "int", nullable: true),
85              BasicSalary = table.Column<decimal>(type: "decimal(8,2)", precision: 8, scale: 2, nullable: true),
86              AgeBonus = table.Column<decimal>(type: "decimal(8,2)", precision: 8, scale: 2, nullable: true),
87              VacationSupport = table.Column<string>(type: "nvarchar(max)", nullable: true),
No issues found
```

Źródło: repozytorium projektu aplikacji webowej

Rysunek 7. Miejsce przechowywania migracji przez Entity Framework Core



Źródło: repozytorium projektu aplikacji webowej

Rysunek 8. Konfiguracja modeli za pomocą narzędzia EF Core - Fluent API

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<User>()
        .Property(u => u.Email)
        .IsRequired();

    modelBuilder.Entity<Role>()
        .Property(r => r.Name)
        .IsRequired();

    modelBuilder.Entity<Ambulance>()            // Fluent API
        .Property(a => a.SideNumber)
        .IsRequired()
        .HasColumnName("Numer boczny");

    modelBuilder.Entity<Employee>(eb =>
    {
        eb.Property(p => p.PESEL).IsRequired().HasColumnName("Numer PESEL");
        eb.Property(p => p.Phone).HasColumnType("varchar(9)");
        eb.Property(p => p.FirstName).HasMaxLength(100);
        eb.Property(p => p.YearsWorked).HasDefaultValue(0);
```

Źródło: repozytorium projektu aplikacji webowej

Fluent API to sposób definiowania konfiguracji modelu EF Core w C#. Umożliwia on programistom bardziej elastyczne definiowanie modelu, szczególnie w przypadkach, gdy domyślne konwencje EF Core nie wystarczają do precyzyjnej konfiguracji encji, ich właściwości i relacji między nimi. Umożliwia programistom konfigurowanie modelu bezpośrednio w kodzie C# z użyciem łańcuchów metod, co pozwala na bardziej intuicyjne i czytelne definiowanie modelu. Przykładowo, w Fluent API można skonfigurować wymagane właściwości encji, definiować relacje między encjami, zmieniać nazwy kolumn, określać indeksy i wiele innych opcji konfiguracji. Fluent API jest opcjonalnym elementem EF Core i nie jest wymagane do definiowania modelu. Jednakże, dla bardziej zaawansowanej konfiguracji modelu, Fluent API może okazać się bardzo przydatne i wygodne[18].

W projekcie relacje związków między encjami zostały określone następująco:

- relacja jeden do jednego pomiędzy „Employee" i „Adress", czyli jeden pracownik może mieć tylko jedno miejsce zamieszkania. Jest to realizowane poprzez użycie metody „HasOne()" i „WithOne()",
- relacje jeden do wielu (jeden pracownik może mieć wiele ubezpieczeń, umów, itp.). Jest to realizowane poprzez użycie metody „HasMany()" i „WithOne()",
- relacje wiele do wielu (wiele pracowników może pracować na wielu karetkach). Jest to realizowane poprzez użycie metody „HasMany()" oraz metody „WithMany()",
- relacja jeden do jednego dotyczy „Ambulance" i „Car" (każda karetka może mieć tylko jeden samochód). Jest to realizowane przez użycie metody „HasOne()" i „WithOne()",
- relacje wiele do wielu między „Ambulance" a „Equipment", „DrugType" i „Tool". Każda karetka może mieć wiele urządzeń medycznych, rodzajów leków i narzędzi. Jest to realizowane przez użycie metody „HasMany()" oraz metody „WithMany()",
- relacje wiele do jednego między „Base", „Ambulance" i „Reserve". Każda baza może mieć wiele karetek i rezerw (dodatkowego sprzętu). Jest to realizowane poprzez użycie metody „HasMany()" oraz metod „WithOne()".

[18] https://www.entityframeworktutorial.net/efcore/fluent-api-in-entity-framework-core.aspx [dostęp: 23.02.2023]

Rysunek 9. Zależności między encjami dla klasy "Employee"

```
modelBuilder.Entity<Employee>(eb =>
{
    eb.HasOne(p => p.Adress)
    .WithOne(a => a.Employee)
    .HasForeignKey<Adress>(a => a.EmployeeId);

    eb.HasMany(p => p.Insurances)
    .WithOne(u => u.Employee)
    .HasForeignKey(u => u.EmployeeId);

    eb.HasMany(p => p.Agreements)
    .WithOne(u => u.Employee)
    .HasForeignKey(u => u.EmployeeId);

    eb.HasMany(p => p.Educations)
    .WithOne(w => w.Employee)
    .HasForeignKey(w => w.EmployeeId);

    eb.HasMany(p => p.Ambulances)
    .WithMany(a => a.Employees)
    .UsingEntity<Duty>(
        p => p.HasOne(pr => pr.Ambulance)
        .WithMany()
        .HasForeignKey(pr => pr.AmbulanceId),

        p => p.HasOne(pr => pr.Employee)
        .WithMany()
        .HasForeignKey(pr => pr.EmployeeId),

        pr => pr.HasKey(k => new { k.AmbulanceId, k.EmployeeId })
        );
```

Źródło: repozytorium projektu aplikacji webowej

9.3. API systemu

Za komunikację aplikacji webowej „ZRM Help” z bazą danych odpowiada API (Application Programming Interface), tak zwany interfejs programowania aplikacji. Jak już wcześniej zostało wspominanie API tej aplikacji zostało napisane za pomocą .NET Framework, a dokładnie ASP.NET. Podczas tworzenia API aplikacji webowej jednym z ważniejszych początkowych elementów jest określenie sposobu połączenia z bazą danych. W aplikacji „ZRM Help” proces ten został podzielony na 3 etapy. Jako pierwszy element należy, w pliku głównym projektu „Program.cs”, dodać kontekst połączenia z bazą danych. W tym celu należy za pomocą lokalnej zmiennej „builder” dodajemy kolejny serwis/usługę do projektu za pomocą metody „AddDbContext” z parametrem kontekstu bazy danych. Następnie za pomocą metody „UseSqlServer” definiujemy połączenie z bazą Microsoft SQL Server przekazując to metody parametr ciągu połączenia, tak zwany „ConnectionString” i nadając mu określoną nazwę. Następnie w pliku „appsetings.json” lub „appsetings.Development.json” (tak naprawdę w zależności czy nasz projekt znajduje się w wersji produkcyjnej, czy nie) definiujemy

nasz ciąg połączenia informując API o treści „ConnectionString”. Przekazujemy w nim różne parametry, między innymi nazwę serwera, na którym znajduje się baza danych, samą nazwę bazy danych, ewentualnie login i hasło do zalogowania się do sieciowej bazy. Na sam koniec, aby to wszystko mogło sprawnie funkcjonować należy w pliku „.csproj” lub za pomocą menadżera pakietów NuGet dołączyć do projektu oraz zainstalować odpowiednie paczki zależności.

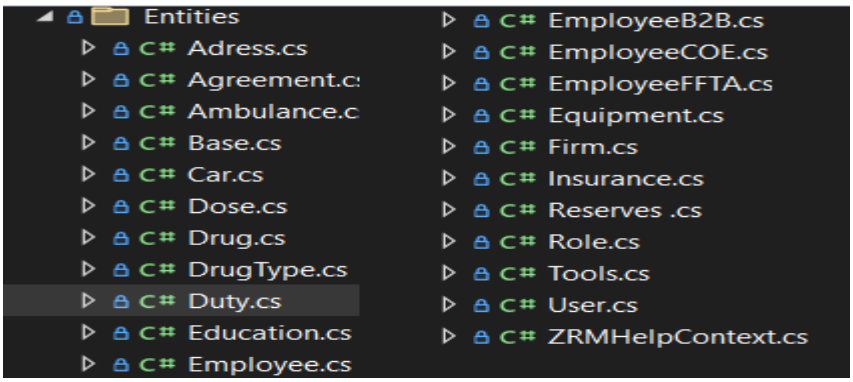
Rysunek 10. Projektowanie połączenia API z bazą danych aplikacji

```
Program.cs
ZRMHelp
12
13    builder.Services.AddDbContext<ZRMHelpContext>(
14            option => option.UseSqlServer(builder.Configuration.GetConnectionString("ZRMHelpDatabase"))
15        );
Program.cs    appsettings.D...elopment.json
Schema: <No Schema Selected>
1  {
2    "ConnectionStrings": {
3      "ZRM_HelpDatabase": "Server=DESKTOP-J26GSV3\\SQLEXPRESS;Database=ZRMHelpDB;Trusted_Connection=True;"
4    },
ZRMHelp    Program.cs    appsettings.Development.json
16
17    <ItemGroup>
18      <PackageReference Include="Microsoft.AspNetCore.SpaProxy" Version="7.0.4" />
19      <PackageReference Include="Microsoft.EntityFrameworkCore" Version="7.0.4" />
20      <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="7.0.4" />
21      <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="7.0.4">
```

Źródło: repozytorium projektu aplikacji webowej

Po ustaleniu połączenia API z bazą danych kolejnym elementem w procesie tworzenia API „ZRM Help” było utworzenie wszystkich klas/encji projektu. W tym celu w głównym katalogu projektu utworzony został folder „Entities”, w któym były umieszczone wszystkie powstające klasy rozwiązania.

Rysunek 11. Encje projektu API aplikacji webowej



Źródło: repozytorium projektu aplikacji webowej

Encje powstawały zgodnie z ogólnie przyjętą norma tworzenia klas w języku C#. Najpierw zdefiniowana została nazwa klasy, a następnie wszystkie jej własności.

Rysunek 12. Przykładowa klasa C# z projektu aplikacji

```
1   namespace ZRMHelp.Entities
2   {
        16 references
3       public class Employee
4       {
            3 references
5           public int Id { get; set; }
            1 reference
6           public string FirstName { get; set; }
            0 references
7           public string LastName { get; set; }
            1 reference
8           public string PESEL { get; set; }
            1 reference
9           public string Phone { get; set; }
            0 references
10          public string Email { get; set; }
            0 references
11          public string Position { get; set; }
            0 references
12          public string Department { get; set; }
            0 references
13          public DateTime StartDate { get; set; }
```

Źródło: repozytorium projektu aplikacji webowej

Do weryfikacji poprawności funkcjonowania API i jej dokumentacji używa się specjalnych aplikacji takich jak: Postman, czy Swagger. O ile Postman nie wymaga żadnej konfiguracji, to jest jednak oddzielna aplikacja, którą trzeba zainstalować na urządzeniu w celu weryfikacji API. Swagger natomiast wymaga pewnej konfiguracji w kodzie API, ale za to jest dostępny z okna przeglądarki, dokładnie pod tym samym adresem URL co API, lecz z dopiskiem „/swagger”. Z tego powodu do projektu został dołączony Swagger, to rozwiązanie wydaje się dużo prostsze, ale przede wszystkim łatwiejsze to utrzymania[19].

[19] https://swagger.io/ [dostęp: 23.02.2023]

Rysunek 13. Dołączanie do projektu dokumentacji API za pomocą Swagger

```
11      builder.Services.AddSwaggerGen();

21      app.UseSwagger();
22      app.UseSwaggerUI(/*c
=> c.SwaggerEndpoint("/swagger/v1/swagger.json",
                                  "ZRMHelp v1")*/);
```

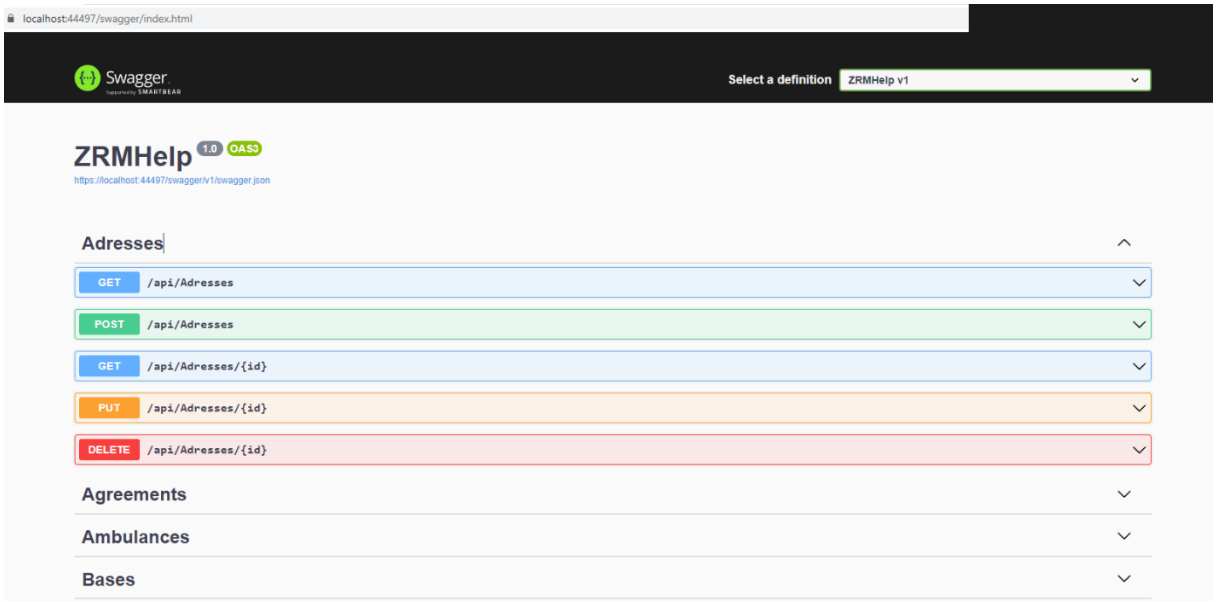
Źródło: repozytorium projektu aplikacji webowej

Za pomocą zmiennej „builder” dodajemy serwis o nazwie metody „AddSwaggerGen”. Następnie za pomocą zmiennej „app”, która tak naprawdę jest zdefiniowana w projekcie jako zmienna „builder” z metodą „Build()”, dołączamy do aplikacji zależność interfejsu użytkowania aplikacji Swagger. Na sam koniec, aby nasz program miał wszystkie potrzebne narzędzia do uruchomienia Swagger musimy dodać kolejna paczkę zależności o nazwie „Swashbuckle.AspNetCore". Za pomocą narzędzia Swagger możemy testować kontrolery naszego API. W jego głównym oknie pojawiają się wszystkie działające kontrolery, wraz z przypisanymi do nich metodami HTTP. Cztery podstawowe metody każdego kontrolera API to:

- GET – zwraca/pobiera wartość obiektu, jakiego adres został dostarczony,
- POST – dodaje/przesyła do bazy danych podaną wartość,
- PUT – aktualizuje element rekordu do bazy przez podaną wartość,
- DELETE – usuwa dany element lub dany rekord[20].

[20] https://kobietydokodu.pl/niezbednik-juniora-protokol-http/ [dostęp: 23.02.2023]

Rysunek 14. Widok strony głównej dokumentacji API za pomocą Swagger



Źródło: repozytorium projektu aplikacji webowej

Kontrolery odpowiedzialne są za zarządzane danymi bazy danych, a raczej wysyłaniu poleceń do bazy danych zgodnie z podanymi instrukcjami, a następnie zwracają nam otrzymaną odpowiedź od serwera. Skoro API odpowiada za komunikację interfejsu użytkownika z serwerem to można powiedzieć, że kontrolery są swojego rodzaju API dla samego API.

Rysunek 15. Metoda GET do zwracania wszystkiego oraz GET z parametrem "id"

```
23          // GET: api/Adresses
24          [HttpGet]
            0 references
25          public async Task<ActionResult<IEnumerable<Adress>>> GetAdresses()
26          {
27              return await _context.Adresses.ToListAsync();
28          }
29
30          // GET: api/Adresses/5
31          [HttpGet("{id}")]
            0 references
32          public async Task<ActionResult<Adress>> GetAdress(int id)
33          {
34              var adress = await _context.Adresses.FindAsync(id);
35
36              if (adress == null)
37              {
38                  return NotFound();
39              }
40
41              return adress;
42          }
```

Źródło: repozytorium projektu aplikacji webowej

Najbardziej podstawową metodą HTTP kontrolera jest metoda GET nie przekazująca żadnego parametru z definicji. Taka metoda zwraca listę wszystkich elementów w danej tabeli. Zgodnie z rysunkiem numer 37 cała metoda GET polega na stworzeniu publicznej metody, w przykładzie dodatkowo asynchronicznej, typu „ActionResult”, do której przekazujemy kolekcje określonego typu, w tym przypadku typu „Adress”. Metoda zwraca nam prywatny obiekt „_context” z właściwością „Adresses” oraz „ToListAsync()”, co w zasadzie wraca nam wszystkie obiekty danej tabeli w formie listy. Specjalnym przypadkiem tej metody jest GET z przekazywanym parametrem „id”. Jedyna różnica polega na tym, że metoda otrzymuje zapytanie z zawartością (tak zwanym parametrem), przez co najpierw sprawdza za pomocą funkcji „if”, czy element z danym parametrem istnieje w bazie danych. W przypadku pozytywnej odpowiedzi system zwraca tą wartość, a w przeciwnym wypadku otrzymujemy komunikat generowany przez metodę „NotFound()” o braku możliwości zlokalizowania pożądanego przez nas elementu.

Rysunek 16. Metoda PUT kontrolera do zarzadzania adresami

```
46          [HttpPut("{id}")]
            0 references
47          public async Task<IActionResult> PutAdress(int id, Adress adress)
48          {
49              if (id != adress.Id)
50              {
51                  return BadRequest();
52              }
53
54              _context.Entry(adress).State = EntityState.Modified;
55
56              try
57              {
58                  await _context.SaveChangesAsync();
59              }
60              catch (DbUpdateConcurrencyException)
61              {
62                  if (!AdressExists(id))
63                  {
64                      return NotFound();
65                  }
66                  else
67                  {
68                      throw;
69                  }
70              }
71
72              return NoContent();
```

Źródło: repozytorium projektu aplikacji webowej

Metoda PUT natomiast mówi się potocznie, że aktualizuje konkretny rekord w bazie danych. Natomiast to co robi tak naprawdę to podmienia rekord w bazie danych o podanym parametrze z nowym rekordem przez nas podanym, a w przypadku nie znalezienia danego rekordu powinien utworzyć w tym miejscu nowy.

Zasadnicza różnica między PUT, a aktualizacją zasobu polega na tym, że PUT aktualizuje cały zasób bez znaczenia jakie dokładnie przekazaliśmy parametry, w pewien sposób go podmienia, a parametry których nie podaliśmy zostaną puste. Nie jest możliwa częściowa aktualizacja zasobu za pomocą metody PUT.

Rysunek 17. Metody POST oraz DELETE z przykładowego kontrolera

```
77      [HttpPost]
        0 references
78      public async Task<ActionResult<Adress>> PostAdress(Adress adress)
79      {
80          _context.Adresses.Add(adress);
81          await _context.SaveChangesAsync();
82
83          return CreatedAtAction("GetAdress", new { id = adress.Id }, adress);
84      }
85
86      // DELETE: api/Adresses/5
87      [HttpDelete("{id}")]
        0 references
88      public async Task<IActionResult> DeleteAdress(int id)
89      {
90          var adress = await _context.Adresses.FindAsync(id);
91          if (adress == null)
92          {
93              return NotFound();
94          }
95
96          _context.Adresses.Remove(adress);
97          await _context.SaveChangesAsync();
98
99          return NoContent();
100     }
```

Źródło: repozytorium projektu aplikacji webowej

Metoda POST tworzy nowy zasób na podstawie przekazanego parametru, a następnie zwraca go z adnotacją metody „CreatedAction”. Metoda DELETE po prostu usuwa podanych przez nas zasób. Przekazywany jest do niej parametr „id” danego elementy tabeli, a on najpierw sprawdza, czy taki zasób istnieje. W przypadku pozytywnego zlokalizowania zasobu usuwa go za pomocą metody „Remove(zasób)”.

Funkcjonowanie kontrolerów w API nie byłoby możliwe, gdyby nie LINQ. To właśnie on udostępnia możliwość formułowania zapytań na poziomie języka. LINQ oferuje spójny model charakteru pracy z danymi. Przy użyciu zapytań LINQ zawsze pracuje się z obiektami. Tworzenie zapytań przy użyciu LINQ pozwala na wydzielenie pewnej części operowanych obiektów według zadeklarowanych potrzeb. Dokładnie tak samo jak to ma przy zastosowaniu języka SQL. Wprowadzane zostają warunki, a następnie operacje na kolekcjach obiektów. Przez to można powiedzieć, że LINQ jest specjalnym wystąpieniem języka SQL w środowisku języka programowania jakim jest C#, a w konsekwencji środowiska firmy Microsoft.

9.4. Graficzny interfejs użytkownika

Tworzenie interfejsu użytkownika aplikacji webowej za pomocą React należy zacząć od zwrócenia szczególnej uwagi na pliki źródłowe dostarczone przez szablon „create-react-app”. Wyświetlenie naszego projektu w przeglądarce internetowej zawdzięczany tak naprawdę jednemu, niepozornemu plikowi „index.html”. W jego treści, w sekcji „body”, znajduje się tak naprawdę tylko jeden element, który pozostanie tym jednym elementem na zawsze. Należałoby się zastanowić, jak więc to możliwe, że tak zawansowane projektu w React wyświetlane są za pomocą jednego elementu określonego w znacznikach HTML.

Rysunek 18. Sekcja "body" pliku index.html projektu aplikacji

```
14    <body>
15      <noscript>
16        You need to enable JavaScript to run this app.
17      </noscript>
18      <div id="root"></div>
19
20    </body>
21  </html>
```

Źródło: repozytorium projektu aplikacji webowej

Jest to element typu „div” o określonym parametrze „id” wynoszącym „root”. Wszystko to co robi ten obiekt to odbiera od aplikacji element o „id=root” i wyświetla go w body strony. Element ten określony jest w kolejnym ramowym elemencie każdego projektu, w pliku „index.js”. W tym pliku określany jest podstawowy adres URL, a przede wszystkim określona zostaje budowa elementu „root”.

Rysunek 19. Element "root" w pliku index.js

```
root.render(
  <BrowserRouter basename={baseUrl}>
    <App />
  </BrowserRouter>);
```

Źródło: repozytorium projektu aplikacji webowej

Tutaj dowiadujemy się, że element „root” odpowiedzialny jest za wyświetlenie elementu „App”, który to z kolei przekazany jest za pomocą pliku „App.js”. Plik ten to tak naprawdę serce całego projektu aplikacji webowej pisanej za pomocą React.

Rysunek 20. Zawartość pliku źródłowego App.js

```
App.js
src > App.js > ...
You, yesterday | 1 author (You)
1  import React, { Component } from 'react';
2  import { Route, Routes } from 'react-router-dom';
3  import AppRoutes from './AppRoutes';
4  import { Layout } from './components/Layout';
5  import './custom.css';
6  import store from './store'
7  import { Provider } from 'react-redux'        You, 2 days ago • main fix …
8  import Footer from './Footer';
9
   You, yesterday | 1 author (You)
10 export default class App extends Component {
11   static displayName = App.name;
12
13   render() {
14     return (
15       <Provider store={store}>
16         <Layout>
17           <Routes>
18             {AppRoutes.map((route, index) => {
19               const { element, ...rest } = route;
20               return <Route key={index} {...rest} element={element} />;
21             })}
22           </Routes>
23           <Footer />
24         </Layout>
25       </Provider>
26     );
27   }
28 }
```

Źródło: repozytorium projektu aplikacji webowej

Do pliku „App.js” importowane są wszystkie inne komponenty z projektu React, a sam eksportuje klasę „App”. Jest to bez wątpienia jeden z najważniejszych plików projektu. Za pomocą metody „render” zwracamy obiekt typu JSX, który renderuje komponent „Provider” z React Redux, który dostarcza globalne stany aplikacji do wszystkich komponentów potomnych. Jako właściwość „store” przekazujemy obiekt „store”, który jest importowany z pliku ./store. Wewnątrz komponentu Provider znajduje się Layout, który renderuje naszą aplikację i podłącza React Router. Elementy Route, które definiują poszczególne ścieżki URL, są mapowane z AppRoutes. Ten kod jest głównym plikiem aplikacji React, który renderuje stronę internetową i obsługuje routowanie.

Rysunek 21. Routing projektu aplikacji webowej

```
const AppRoutes = [
  {
    index: true,
    element: <Login />
  },
  {
    path: '/signup',
    element: <SignUp />
  },
  {
    path: '/home',
    element: <LoggedIn><Home /></LoggedIn>
  },
  {
    path: '/drugs',
    element: <LoggedIn><Drugs /></LoggedIn>
  },
  {
    path: '/employees',
    element: <LoggedIn><Employees /></LoggedIn>
  }
];

export default AppRoutes;
```

Źródło: repozytorium projektu aplikacji webowej

W tym pliku kod definiuje tablicę AppRoutes, która zawiera konfigurację ścieżek URL i komponentów do renderowania dla każdej z tych ścieżek. Pierwszy obiekt w tablicy oznaczony kluczem "index: true" definiuje, że ta ścieżka jest ścieżką domową, tak zwanym „landing page”.

Elementem, który ma być renderowany dla tej ścieżki, jest komponent Login z pliku „Login.js”, który prezentuje widok strony logowania aplikacji. Elementy „<LoggedIn>” są używane do renderowania komponentów, które wymagają, aby użytkownik był zalogowany. Wszystkie komponenty zdefiniowane wewnątrz tych elementów zostaną wyrenderowane tylko wtedy, gdy użytkownik będzie zalogowany. Zgodnie z czym jedyne widoki, które może zobaczyć niezalogowany użytkownik to „Login” i „SignUp”, z czego ten drugi służy jako formularz do rejestracji nowego użytkownika.

Następnie w komponencie „LoggenIn.js” wykorzystuje się hook „useSelector”, który pozwala na pobranie stanu z Redux Store. Funkcja ta przyjmuje funkcję selektora, która zwraca część stanu, która jest potrzebna w komponencie. W tym przypadku selektor zwraca wartość „user” z części stanu o nazwie „user”, która jest przechowywana w Redux Store.

Następnie wykorzystywany jest wynik wywołania selektora do wyświetlenia menu nawigacyjnego „NavMenu” z właściwością „isAdmin”, która otrzymuje wartość „true” lub „false” w zależności od tego, czy użytkownik jest zalogowany jako administrator.

Rysunek 22. Logika komponentu "LoggedIn"

```
export const LoggedIn = (props) => {
  const user = useSelector((state) => state.user.user);

  return (
    <div>
      <NavMenu isAdmin={user?.isAdmin}/>
      {props.children}
    </div>
  );
}
```

Źródło: repozytorium projektu aplikacji webowej

Rysunek 23. Kod źródłowy pliku userSlice.js uzupełniający logikę "LoggedIn"

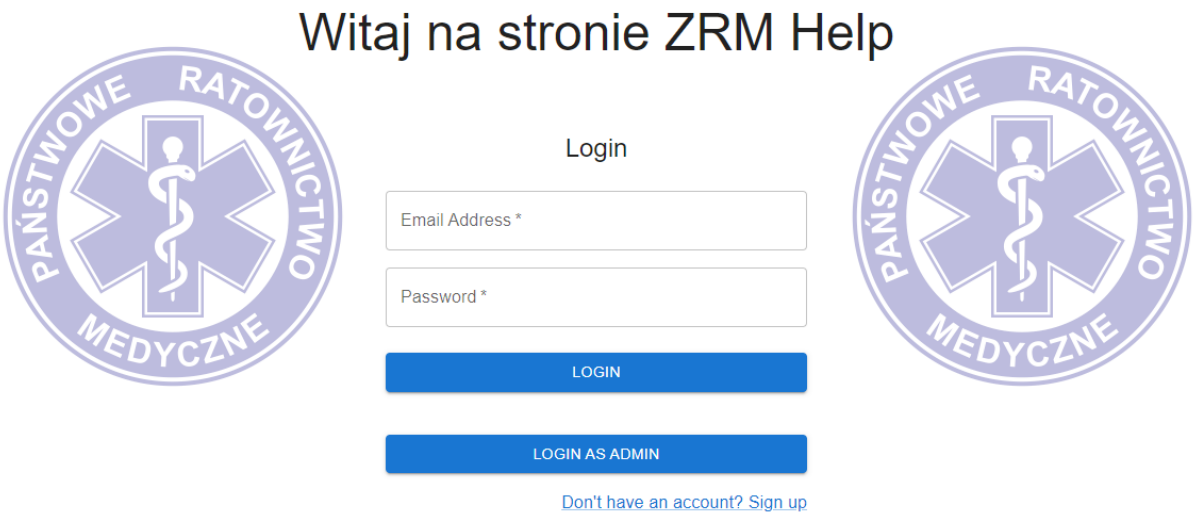
```
3  export const userSlice = createSlice({
4    name: 'user',
5    initialState: {
6      user: {isAdmin: false},
7    },
8    reducers: {
9      logAsAdmin: (state) => {
10       console.log('log adm')
11       state.user = {isAdmin: true}
12     },
13     logAsUser: (state) => {
14       console.log('log usr')
15       state.user = {isAdmin: false}
16     }
17   },
18 })
```

Źródło: repozytorium projektu aplikacji webowej

Rysunek 24. Widok menu nawigacyjnego w zależności od właściwości "isAdmin"

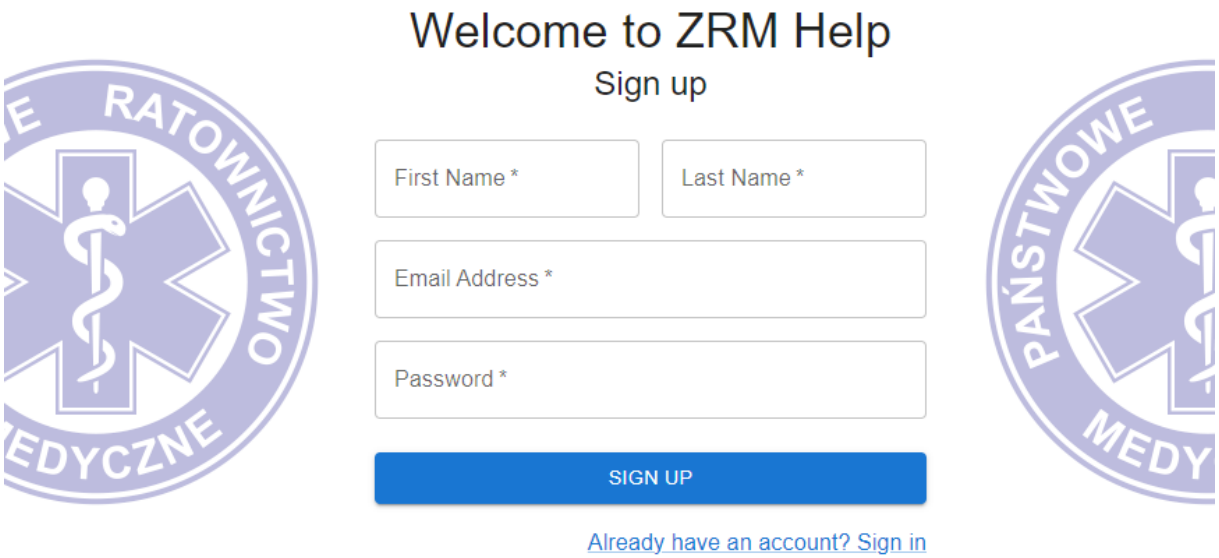
Źródło: repozytorium projektu aplikacji webowej

Rysunek 25. Widok strony logowania aplikacji



Źródło: repozytorium projektu aplikacji webowej

Rysunek 25. Widok strony z formularzem rejestracji aplikacji



Źródło: repozytorium projektu aplikacji webowej

W celu stworzenia widoku strony logowania i strony rejestracji użytkownika użyte zostały gotowe komponenty biblioteki Material UI takie jak: „Avatar”, „Button”, „CssBaseline”, „TextField”, „FormControlLabel”, „Checkbox”, „Link”, „Grid”, „Box”, „Typography” oraz „Container”. „Login” również korzysta z hooków „useDispatch” i „useNavigate”. „useDispatch” jest używany do wysyłania akcji do magazynu Redux, a „useNavigate” jest używany do nawigacji po aplikacji React Router.

Rysunek 26. Przykładowe Hooki zawarte w komponencie Login.js

```
export const Login = () => {
  const dispatch = useDispatch();
  const navigate = useNavigate();
```

Źródło: repozytorium projektu aplikacji webowej

Hooki (ang. "hooks") w React to funkcje, które umożliwiają korzystanie z funkcjonalności React w komponentach funkcyjnych. Zostały one wprowadzone w wersji 16.8 jako uzupełnienie do dotychczasowej klasy komponentów. Korzystanie z hooków pozwala na użycie stanu, efektów ubocznych (ang. "side effects"), kontekstu, refów i innych funkcjonalności w komponentach funkcyjnych. Przykłady hooków w React:

- „useState” - służy do zarządzania stanem w komponentach funkcyjnych,
- „useEffect” - pozwala na korzystanie z efektów ubocznych, takich jak pobieranie danych z API lub manipulowanie DOM-em,
- „useContext” - umożliwia dostęp do kontekstu aplikacji[21].

React Router to biblioteka dla Reacta, która umożliwia nawigację między różnymi widokami (stronami) w aplikacji internetowej. Daje programistom wiele opcji, takich jak: tworzenie dynamicznych ścieżek URL, przekazywanie parametrów i zapytań, obsługa historii przeglądania (poprzez przycisk "Wstecz" przeglądarki), a także wiele innych. React Router składa się z kilku komponentów, takich jak „BrowserRouter”, „Route” i „Link”, które pozwalają na budowanie i zarządzanie ścieżkami w aplikacji. Umożliwia to łatwe przenoszenie użytkownika między różnymi stronami aplikacji bez konieczności przeładowania całej strony[22].

Redux jest biblioteką do zarządzania stanem aplikacji w React. Pozwala na przechowywanie i aktualizowanie globalnego stanu aplikacji w jednym miejscu. Zmiana stanu w Redux jest możliwa tylko poprzez wysłanie akcji (ang. action), której typ jest definiowany przez autora. Akcja jest następnie przekazywana do tzw. „reducer-a”, który na podstawie jej typu i treści dostosowuje stan aplikacji. Stan jest przetrzymywany w magazynie (ang. store), z którego można go pobierać i wykorzystywać w dowolnym miejscu w aplikacji[23].

[21] https://pl.reactjs.org/docs/hooks-overview.html [dostęp: 23.02.2023]

[22] https://reactrouter.com/en/main [dostęp: 23.02.2023]

[23] https://redux.js.org/ [dostęp: 23.02.2023]

Rysunek 27. Logika pobierania informacji o lekach z API

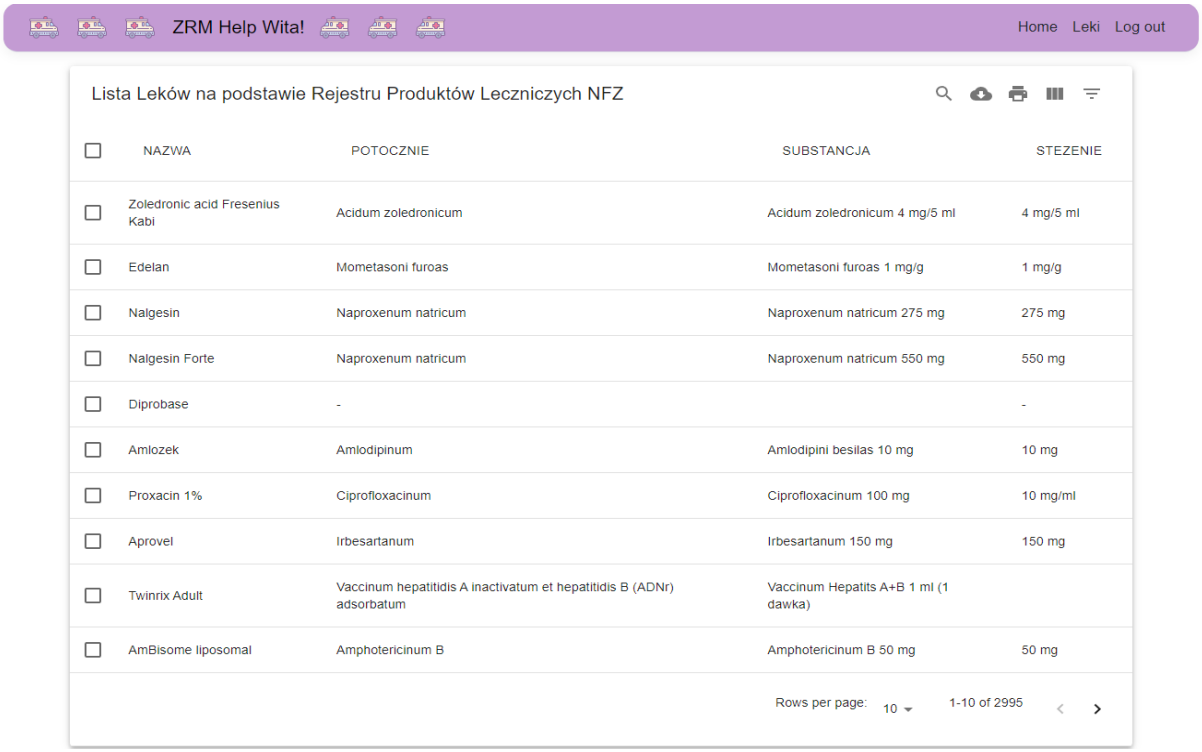
```
const columNames = ['Nazwa', 'Potocznie', 'Substancja', 'Stezenie']

export const Drugs = props => {
  const drugs = useSelector((state) => state.drugs.drugs);
  const dispatch = useDispatch();
  useEffect(() => {
    dispatch(fetchDrugs());
  }, []);
```

Źródło: repozytorium projektu aplikacji webowej

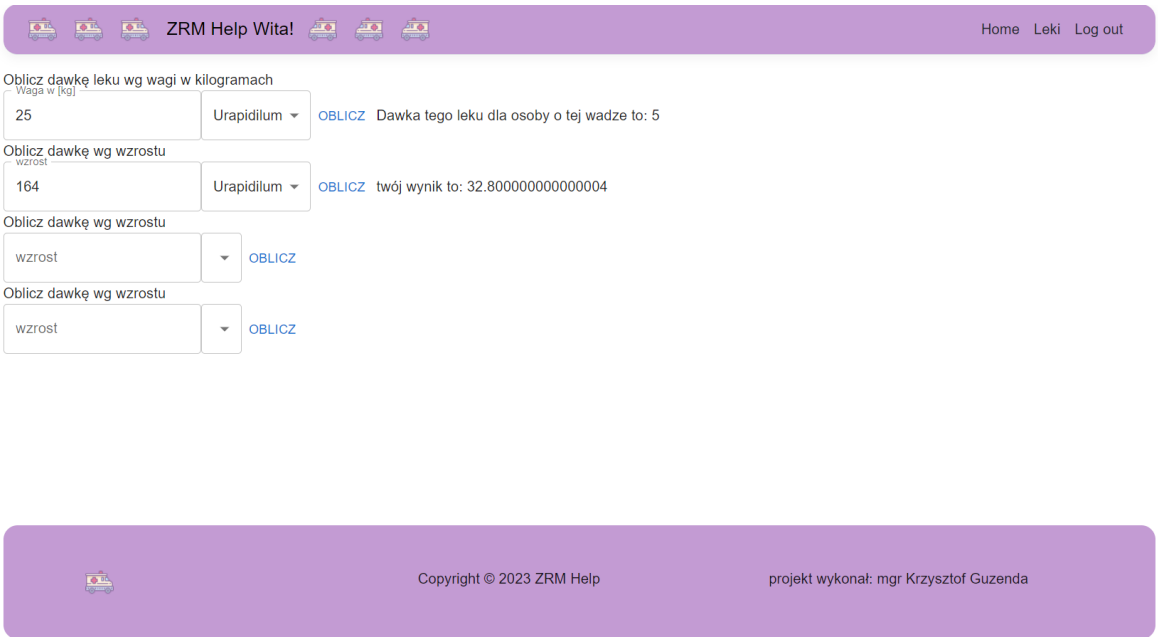
„useSelector” pozwala na pobieranie danych z Redux, pobiera tablicę leków z „state.drugs.drugs” i przypisuje ją do zmiennej „drugs”. „useDispatch” zwraca funkcję do wywoływania akcji Redux, a funkcja ta jest przypisana do zmiennej „dispatch”. „useEffect” umożliwia wykonywanie operacji efektów ubocznych w komponencie funkcyjnym, pobiera dane o lekach przy pierwszym renderowaniu komponentu za pomocą funkcji „dispatch(fetchDrugs())”. Funkcja „fetchDrugs()” jest akcją zdefiniowaną w „drugsSlice”, która pobiera dane o lekach z API i zapisuje je w store'u Redux.

Rysunek 28. Widok strony z wyszukiwarka leków aplikacji



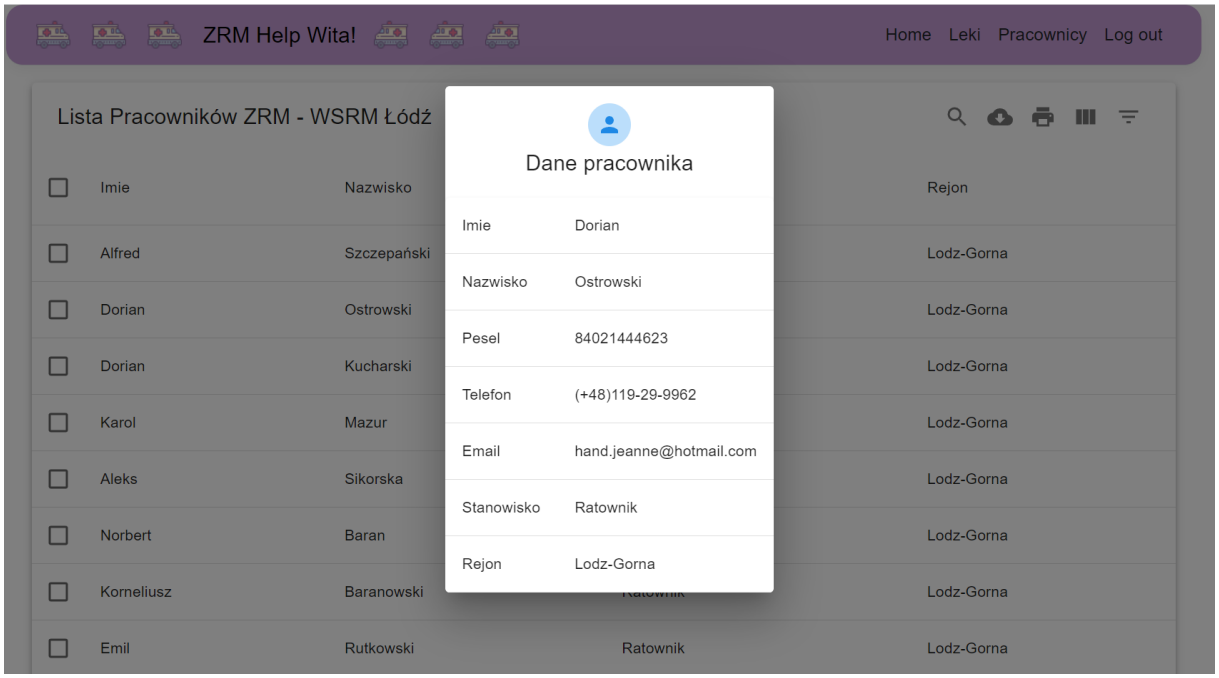
Źródło: repozytorium projektu aplikacji webowej

Rysunek 29. Widok strony z kalkulatorami medycznymi aplikacji



Źródło: repozytorium projektu aplikacji webowej

Rysunek 30. Widok strony z wyszukiwarką pracowników i danymi kontaktowymi



Źródło: repozytorium projektu aplikacji webowej